

Python

Introduzione a Python

Cos'è Python, come funziona e come iniziare a usarlo.

Cos'è Python?

Python è un **linguaggio di programmazione** — cioè un modo per dare istruzioni al computer. È stato creato da **Guido van Rossum** e pubblicato per la prima volta nel **1991**.

La cosa speciale di Python è che le sue istruzioni assomigliano quasi all'italiano o all'inglese semplice. Leggi questo codice:

```
if temperatura > 30:
    print("Fa caldo!")
else:
    print("Temperatura nella norma.")
```

Anche senza sapere nulla di programmazione, capisci cosa fa. Questo è Python.

Dove si usa Python nel mondo reale?

Python non è un linguaggio "per principianti" — è usato professionalmente in tantissimi campi:

- **Siti web:** Instagram, YouTube e Pinterest sono costruiti con Python
 - **Intelligenza artificiale:** ChatGPT, DALL-E e molti altri sistemi AI usano Python
 - **Analisi dati:** i ricercatori e gli scienziati lo usano per elaborare dati
 - **Automazione:** per automatizzare compiti ripetitivi al computer
 - **Giochi e grafica:** per prototipi e strumenti di sviluppo
-

Perché iniziare da Python?

- **Sintassi semplice:** meno simboli strani rispetto ad altri linguaggi
 - **Vedi i risultati subito:** non devi fare passaggi extra per far girare il codice
 - **Funziona ovunque:** su Windows, macOS e Linux
 - **Enorme comunità:** trovi tutorial, risposte e librerie per qualsiasi cosa
 - **Gratis e open source:** nessun costo, nessuna licenza
-

Come funziona Python?

Python è un linguaggio **interpretato**: scrivi il codice, e un programma chiamato **interprete** lo legge ed esegue riga per riga, immediatamente.

È come avere un traduttore in tempo reale: tu parli (scrivi codice), il traduttore (l'interprete) converte le tue parole in azioni che il computer capisce.

Il flusso di lavoro è semplice:

1. Scrivi il codice in un file con estensione `.py`
 2. L'interprete Python legge ed esegue il file
 3. Vedi il risultato nel terminale
-

Installazione

Scarica Python dal sito ufficiale: python.org/downloads

Durante l'installazione su Windows, assicurati di spuntare la casella **"Add Python to PATH"** — altrimenti il terminale non troverà Python.

Verifica che l'installazione sia andata a buon fine aprendo il terminale e digitando:

```
python --version
```

Dovresti vedere qualcosa come `Python 3.12.0`.

Il primo programma: "Hello, World!"

Per tradizione, il primo programma che si scrive in qualsiasi linguaggio saluta il mondo. In Python è una sola riga:

```
print("Hello, World!")
```

Salva questo codice in un file chiamato `hello.py`. Poi apri il terminale nella stessa cartella e digita:

```
python hello.py
```

Output:

```
Hello, World!
```

La funzione `print()` mostra testo sullo schermo. Sarà una delle cose che userai di più — specialmente all'inizio, per vedere i risultati del tuo codice.

La console interattiva (REPL)

Python ha anche una modalità interattiva: digita `python` nel terminale (senza indicare un file) e puoi scrivere istruzioni una alla volta, vedendo subito il risultato.

```
>>> print("Ciao!")
Ciao!
>>> 2 + 3
5
>>> nome = "Alice"
>>> print("Ciao,", nome)
Ciao, Alice
```

Le `>>>` indicano che Python è in attesa di un'istruzione. Questa modalità è ottima per fare esperimenti rapidi e testare piccole idee senza dover creare un file.

Da dove continuare

Se stai iniziando ora, segui queste guide in ordine:

1. Sintassi di base
2. Variabili
3. Tipi di dati
4. Operatori
5. Condizioni
6. Cicli for
7. Liste
8. Funzioni

Quando questi concetti sono chiari, puoi passare a [classi e oggetti](#), [moduli](#) e [gestione dei file](#).

Dizionari avanzati

Tecniche avanzate per lavorare con i dizionari in Python.

Perché ti servono i dizionari avanzati

I dizionari di base li conosci già. Queste tecniche avanzate ti permettono di **crearli in modo più compatto**, gestire chiavi mancanti senza errori, contare occorrenze, e molto altro — cose che tornano utili continuamente nella programmazione reale.

Dictionary comprehension: creare dizionari in una riga

Esattamente come le list comprehension, ma il risultato è un dizionario:

```
# Crea un dizionario {numero: suo_quadrato} per i numeri da 1 a 5
quadrati = {x: x**2 for x in range(1, 6)}
print(quadrati)    # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# Crea un dizionario da due liste
nomi = ["Alice", "Bob", "Carlo"]
voti = [8, 7, 9]
registro = {nome: voto for nome, voto in zip(nomi, voti)}
print(registro)    # {'Alice': 8, 'Bob': 7, 'Carlo': 9}

# Con un filtro: solo i pari e i loro quadrati
pari_quadrati = {x: x**2 for x in range(10) if x % 2 == 0}
print(pari_quadrati)    # {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
```

get() : accedere senza rischio di errore

Se accedi a una chiave che non esiste con `dizionario["chiave"]`, Python lancia un `KeyError`. Il metodo `get()` restituisce invece un valore di default.

```
studenti = {"Alice": 8, "Bob": 7}

print(studenti.get("Alice"))    # 8      ← chiave trovata
print(studenti.get("Carlo"))    # None   ← chiave non trovata, nessun
errore
print(studenti.get("Carlo", "N/A")) # N/A   ← chiave non trovata, usa il
valore di default
```

setdefault() : aggiungi una chiave solo se non esiste

Restituisce il valore se la chiave c'è già, altrimenti la aggiunge con il valore di default e restituisce quello.

```
dizionario = {"a": 1}

valore = dizionario.setdefault("a", 99)  # "a" esiste già → non cambia
nulla
print(valore)      # 1
print(dizionario)  # {'a': 1}

valore = dizionario.setdefault("b", 99)  # "b" non esiste → la aggiunge
print(valore)      # 99
print(dizionario)  # {'a': 1, 'b': 99}
```

Un uso pratico: costruire un dizionario dove i valori sono liste, senza dover controllare se la chiave esiste già:

```
# Raggruppa le parole per la loro prima lettera
parole = ["mela", "banana", "more", "mandarino"]
raggruppate = {}

for parola in parole:
    lettera = parola[0]
    raggruppate.setdefault(lettera, []).append(parola)  # Se la chiave non
c'è, crea una lista vuota

print(raggruppate)
# {'m': ['mela', 'more', 'mandarino'], 'b': ['banana']}
```

Counter : conta le occorrenze automaticamente

Dal modulo `collections`, `Counter` è un dizionario speciale pensato per contare quante volte appare ogni elemento.

```

from collections import Counter

# Conta quante volte appare ogni frutto
frutta = ["mela", "banana", "mela", "uva", "banana", "mela"]
contatore = Counter(frutta)
print(contatore)
# Counter({'mela': 3, 'banana': 2, 'uva': 1})

print(contatore["mela"])      # 3 – occorrenze di "mela"
print(contatore["kiwi"])      # 0 – chiave non trovata → 0 (non
                               # errore!)
print(contatore.most_common(2)) # [('mela', 3), ('banana', 2)] – i 2 più
                               # frequenti

# Conta le lettere in una stringa
lettere = Counter("mississippi")
print(lettere)
# Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})

```

defaultdict : valori di default automatici

Un altro strumento di `collections`. Con `defaultdict` non devi mai preoccuparti di controllare se una chiave esiste: se non c'è, viene creata automaticamente con il valore di default che hai scelto.

```

from collections import defaultdict

# defaultdict(list) crea automaticamente una lista vuota per le chiavi
nuove
gruppi = defaultdict(list)

studenti = [("Alice", "3A"), ("Bob", "3B"), ("Carlo", "3A")]
for nome, classe in studenti:
    gruppi[classse].append(nome)  # Non serve controllare se "3A" esiste
    già!

print(dict(gruppi))
# {'3A': ['Alice', 'Carlo'], '3B': ['Bob']}

```

Unire due dizionari

```
d1 = {"a": 1, "b": 2}
d2 = {"b": 3, "c": 4}

# Python 3.9+: operatore | (come l'unione degli insiemi)
unito = d1 | d2
print(unito)    # {'a': 1, 'b': 3, 'c': 4} ← "b" di d2 sovrascrive "b" di d1

# Alternativa (funziona in tutte le versioni)
unito2 = {**d1, **d2}
print(unito2)   # {'a': 1, 'b': 3, 'c': 4}
```

Invertire un dizionario (chiavi ↔ valori)

```
originale = {"a": 1, "b": 2, "c": 3}
invertito = {v: k for k, v in originale.items()} # Scambia chiavi e valori
print(invertito)    # {1: 'a', 2: 'b', 3: 'c'}
```

Attenzione: funziona solo se i valori sono unici. Se due chiavi avevano lo stesso valore, una verrà sovrascritta.

Ordinare un dizionario

I dizionari in Python 3.7+ mantengono l'ordine di inserimento. Ma se vuoi ordinarli per chiave o per valore:

```
voti = {"Alice": 8, "Carlo": 9, "Bob": 7}

# Ordina per chiave (alfabetico)
per_nome = dict(sorted(voti.items()))
print(per_nome)    # {'Alice': 8, 'Bob': 7, 'Carlo': 9}

# Ordina per valore, dal più alto al più basso
per_voto = dict(sorted(voti.items(), key=lambda x: x[1], reverse=True))
print(per_voto)    # {'Carlo': 9, 'Alice': 8, 'Bob': 7}
```

Liste avanzate

List comprehension e tecniche avanzate per le liste in Python.

Perché è utile usare le liste avanzate

Sai già usare le liste di base. Ora impari le tecniche che rendono il codice Python **più corto, più leggibile e più efficiente** — strumenti che i programmatori Python usano ogni giorno.

List comprehension: creare liste in una riga

La **list comprehension** è un modo compatto per costruire una lista a partire da un'altra sequenza, tutto in una sola riga.

Il problema che risolve

```
# Metodo tradizionale: 4 righe per creare una lista di quadrati
quadrati = []
for x in range(1, 6):
    quadrati.append(x ** 2)
print(quadrati)    # [1, 4, 9, 16, 25]

# Con list comprehension: 1 sola riga, stesso risultato
quadrati = [x ** 2 for x in range(1, 6)]
print(quadrati)    # [1, 4, 9, 16, 25]
```

Leggi la list comprehension come: "x al quadrato, per ogni x in range(1, 6)".

Con un filtro (**if**)

Puoi aggiungere una condizione per includere solo certi elementi:

```
# Solo i numeri pari tra 0 e 9
pari = [x for x in range(10) if x % 2 == 0]
print(pari)    # [0, 2, 4, 6, 8]

# Solo le parole di più di 4 lettere
parole = ["mela", "banana", "uva", "ciliegia", "kiwi"]
lunghe = [p for p in parole if len(p) > 4]
print(lunghe)    # ['banana', 'ciliegia']
```

Con **if-else** (trasformazione condizionale)

Puoi trasformare ogni elemento in modo diverso a seconda di una condizione:

```
# Classifica ogni numero come "pari" o "dispari"
numeri = range(6)
classificati = ["pari" if x % 2 == 0 else "dispari" for x in numeri]
print(classificati)  # ['pari', 'dispari', 'pari', 'dispari', 'pari', 'dispari']
```

List comprehension annidate (per le matrici)

Puoi usare una list comprehension dentro un'altra per lavorare con strutture a griglia (righe e colonne):

```
# Creare una matrice 3x3 (lista di liste)
matrice = [[j for j in range(3)] for i in range(3)]
print(matrice)  # [[0, 1, 2], [0, 1, 2], [0, 1, 2]]

# "Appiattare" una matrice in una lista semplice
matrice = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
piatta = [elem for riga in matrice for elem in riga]
print(piatta)  # [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Ordinare con `sorted()` e una chiave personalizzata

Per default `sorted()` ordina in ordine alfabetico o numerico. Con il parametro `key` puoi scegliere tu **in base a cosa** ordinare.

```

studenti = [("Alice", 16), ("Carlo", 15), ("Bob", 17)]

# Ordina per età (il secondo elemento della coppia)
per_eta = sorted(studenti, key=lambda s: s[1])
print(per_eta)  # [('Carlo', 15), ('Alice', 16), ('Bob', 17)]

# Ordina per nome (il primo elemento della coppia)
per_nome = sorted(studenti, key=lambda s: s[0])
print(per_nome)  # [('Alice', 16), ('Bob', 17), ('Carlo', 15)]

# Ordine dal più grande al più piccolo
decescente = sorted(studenti, key=lambda s: s[1], reverse=True)
print(decescente)  # [('Bob', 17), ('Alice', 16), ('Carlo', 15)]

```

map() e filter() : applicare funzioni a liste

```

numeri = [1, 2, 3, 4, 5]

# map(): applica una funzione a ogni elemento e restituisce i risultati
doppi = list(map(lambda x: x * 2, numeri))
print(doppi)  # [2, 4, 6, 8, 10]

# filter(): tieni solo gli elementi che soddisfano la condizione
grandi = list(filter(lambda x: x > 3, numeri))
print(grandi)  # [4, 5]

```

:::note Spesso le list comprehension sono più leggibili di `map` e `filter`. Usa quella forma che trovi più chiara. :::

zip() : scorrere più liste insieme

`zip()` combina due o più liste in coppie (o triplete, ecc.), permettendoti di scorrerle tutte insieme.

```
nomi = ["Alice", "Bob", "Carlo"]
voti = [8, 7, 9]
citta = ["Roma", "Milano", "Napoli"]

# Scorrere due liste in parallelo
for nome, voto in zip(nomi, voti):
    print(f"{nome}: {voto}")
# → Alice: 8
# → Bob: 7
# → Carlo: 9

# Creare una lista di coppie
coppie = list(zip(nomi, voti, citta))
print(coppie)
# → [('Alice', 8, 'Roma'), ('Bob', 7, 'Milano'), ('Carlo', 9, 'Napoli')]
```

Operazioni comuni sulle liste

```
numeri = [3, 1, 4, 1, 5, 9, 2, 6]

print(sum(numeri))      # 31 – somma di tutti i valori
print(max(numeri))      # 9 – il valore più grande
print(min(numeri))      # 1 – il valore più piccolo
print(len(numeri))      # 8 – quanti elementi ci sono

# Calcolare la media
media = sum(numeri) / len(numeri)
print(f"Media: {media:.2f}") # Media: 3.88

# Rimuovere i duplicati mantenendo l'ordine originale
unici = list(dict.fromkeys(numeri))
print(unici) # [3, 1, 4, 5, 9, 2, 6]
```

Stringhe avanzate

Metodi avanzati per la manipolazione delle stringhe in Python.

Qual è la funzione delle stringhe avanzate

Le stringhe sono ovunque: nomi, messaggi, dati da file, risposte dal web. Python ha decine di strumenti pronti per cercare, modificare, tagliare e unire testo. In questa pagina vediamo quelli più utili nella pratica.

Cercare testo dentro una stringa

`find()` — trova la posizione di una sottostringa

```
testo = "Ciao, mi chiamo Alice e vivo a Roma."

print(testo.find("Alice"))    # 15 → "Alice" inizia alla posizione 15
print(testo.find("Mario"))    # -1 → non trovato (restituisce -1 invece di
                               # errore)
print(testo.rfind("a"))       # 31 → posizione dell'ULTIMA "a"
```

`count()` — conta quante volte appare qualcosa

```
testo = "banana"
print(testo.count("a"))      # 3
print(testo.count("na"))     # 2
```

`replace()` — sostituisci una parte del testo

```
testo = "Ciao mondo! Ciao Python!"
print(testo.replace("Ciao", "Salve"))      # Salve mondo! Salve Python!
print(testo.replace("Ciao", "Salve", 1))   # Salve mondo! Ciao Python!
(solo la prima)
```

Tagliare e unire

`split()` — divide una stringa in una lista di pezzi

```
frase = "mela banana ciliegia uva"
parole = frase.split()                  # Divide sugli spazi (di default)
print(parole)    # ['mela', 'banana', 'ciliegia', 'uva']

# Dividere su un altro separatore
dati = "Alice,16,Roma"
campi = dati.split(",")                 # Divide sulle virgole
print(campi)    # ['Alice', '16', 'Roma']

# Limitare il numero di divisioni
print("a:b:c:d".split(":", 2))          # ['a', 'b', 'c:d'] ← si ferma dopo 2
tagli
```

`splitlines()` — divide su ogni fine riga

```
testo = "Prima riga\nSeconda riga\nTerza riga"
righe = testo.splitlines()
print(righe)    # ['Prima riga', 'Seconda riga', 'Terza riga']
```

`join()` — unisce una lista di stringhe in una sola

`join()` è l'opposto di `split()` : prende una lista e la incolla insieme usando un separatore.

```
parole = ["mela", "banana", "ciliegia"]
print(", ".join(parole))    # mela, banana, ciliegia
print(" - ".join(parole))  # mela - banana - ciliegia
print("".join(parole))     # melebananaciliegia
```

Pulizia del testo

Rimuovere spazi in eccesso

```
testo = "    ciao mondo    "
print(testo.strip())    # "ciao mondo"    ← rimuove da entrambi i lati
print(testo.lstrip())   # "ciao mondo    " ← rimuove solo a sinistra
print(testo.rstrip())   # "    ciao mondo" ← rimuove solo a destra

# Rimuovere caratteri specifici (non solo spazi)
testo2 = "###ciao###"
print(testo2.strip("#")) # "ciao"
```

Cambiare maiuscole e minuscole

```
testo = "ciao mondo"
print(testo.upper())      # CIAO MONDO
print(testo.lower())      # ciao mondo
print(testo.capitalize()) # Ciao mondo ← solo la prima lettera
                           maiuscola
print(testo.title())      # Ciao Mondo ← prima lettera di ogni parola
                           maiuscola
print(testo.swapcase())   # CIAO MONDO ← inverte: minuscole↔maiuscole
```

Verificare il contenuto di una stringa

Questi metodi restituiscono `True` o `False` :

```
print("hello".islower())   # True ← tutto minuscolo
print("HELLO".isupper())   # True ← tutto maiuscolo
print("Hello World".istitle()) # True ← ogni parola inizia con maiuscola

print("abc123".isalnum())  # True ← solo lettere e numeri
print("abc".isalpha())     # True ← solo lettere
print("123".isdigit())     # True ← solo cifre
print(" ".isspace())       # True ← solo spazi
```

Allineamento e riempimento

Utili per creare tabelle o output ben formattato:

```

print("42".zfill(5))          # "00042"      ← riempie con zeri a
sinistra
print("ciao".ljust(10))       # "ciao      " ← allinea a sinistra (spazi
a destra)
print("ciao".rjust(10))       # "      ciao" ← allinea a destra (spazi a
sinistra)
print("ciao".center(10))      # "   ciao   " ← centra
print("ciao".center(10, "-")) # "---ciao---" ← centra con carattere
personalizzato

```

Dividere in tre parti con `partition()`

`partition()` divide la stringa in esattamente tre parti: prima del separatore, il separatore stesso, dopo il separatore.

```

testo = "nome: Alice"
prima, sep, dopo = testo.partition(":")
print(prima)    # "nome"
print(sep)      # ":"
print(dopo)     # " Alice"

```

Esempio pratico: leggere dati separati da virgole

Un formato molto comune per i dati è il CSV (valori separati da virgole). Ecco come estrarre le informazioni da una riga:

```
riga_csv = "Alice,16,Roma,8.5"
campi = riga_csv.split(",") # Divide sulle virgole

nome = campi[0].strip()      # Rimuove eventuali spazi
eta = int(campi[1].strip())  # Converte in numero intero
citta = campi[2].strip()
voto = float(campi[3].strip()) # Converte in numero decimale

print(f"{nome} ha {eta} anni, vive a {citta} e ha voto {voto}")
# → Alice ha 16 anni, vive a Roma e ha voto 8.5
```

Array (Liste come array)

Come usare le liste Python come array e quando usare librerie specifiche.

Perché è utile usare gli array in Python

In altri linguaggi come C++ o Java, un **array** è una sequenza di elementi tutti dello stesso tipo, con una posizione numerata che parte da 0. In Python, le **liste** svolgono questo ruolo nella maggior parte dei casi — e sono ancora più flessibili.

Le liste come array

Per uso generico, le liste Python si comportano esattamente come array:

```
# Una sequenza di numeri interi, con indici 0, 1, 2, 3, 4
numeri = [10, 20, 30, 40, 50]

# Accedere a un elemento tramite il suo indice (la posizione)
print(numeri[0])    # 10 ← primo elemento
print(numeri[2])    # 30 ← terzo elemento
print(numeri[-1])   # 50 ← ultimo elemento (si conta da destra)

# Modificare un elemento
numeri[1] = 25
print(numeri)       # [10, 25, 30, 40, 50]
```

Il modulo `array` : più efficiente per grandi quantità di numeri

Python ha un modulo chiamato `array` che offre array più efficienti in memoria rispetto alle liste normali. A differenza delle liste, però, possono contenere solo elementi dello **stesso tipo**.

```
# Crea un array di interi ('i' = integer)
numeri = array.array('i', [1, 2, 3, 4, 5])

print(numeri)          # array('i', [1, 2, 3, 4, 5])
print(numeri[0])       # 1

numeri.append(6)        # Aggiunge un elemento in fondo
print(numeri)          # array('i', [1, 2, 3, 4, 5, 6])

numeri.insert(2, 99)    # Inserisce 99 alla posizione 2
print(numeri)          # array('i', [1, 2, 99, 3, 4, 5, 6])

numeri.remove(99)       # Rimuove la prima occorrenza di 99
print(numeri)          # array('i', [1, 2, 3, 4, 5, 6])
```

I codici tipo più usati:

Codice	Tipo	Descrizione
'i'	int	Numero intero
'l'	long	Numero intero lungo
'f'	float	Numero decimale (singola precisione)
'd'	double	Numero decimale (doppia precisione)

Matrici: array a due dimensioni

Una **matrice** è come una tabella con righe e colonne. In Python si rappresenta come una lista di liste:

```
# Una tabella 3x3
matrice = [
    [1, 2, 3], # riga 0
    [4, 5, 6], # riga 1
    [7, 8, 9]  # riga 2
]

# Accedere a un elemento: prima la riga, poi la colonna
print(matrice[0][0]) # 1 ← riga 0, colonna 0
print(matrice[1][2]) # 6 ← riga 1, colonna 2
print(matrice[2])    # [7, 8, 9] ← tutta la riga 2

# Modificare un elemento
matrice[1][1] = 99
print(matrice[1])    # [4, 99, 6]
```

Scorrere un array (o una matrice) con **for**

```
numeri = [10, 20, 30, 40, 50]

# Scorrere tutti gli elementi
for n in numeri:
    print(n)

# Scorrere con l'indice
for i in range(len(numeri)):
    print(f"numeri[{i}] = {numeri[i]}")

# Scorrere una matrice riga per riga, elemento per elemento
matrice = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
for riga in matrice:
    for elemento in riga:
        print(elemento, end=" ")    # Stampa sulla stessa riga
    print()    # Va a capo dopo ogni riga
```

Operazioni comuni

```
numeri = [5, 2, 8, 1, 9, 3]

print(max(numeri))    # 9 - il valore più grande
print(min(numeri))    # 1 - il valore più piccolo
print(sum(numeri))    # 28 - la somma di tutti
print(len(numeri))    # 6 - quanti elementi ci sono
print(sorted(numeri)) # [1, 2, 3, 5, 8, 9] - copia ordinata
```

Quando usare NumPy invece delle liste?

Le liste Python vanno bene per la maggior parte degli usi. Ma se devi fare **calcoli matematici su grandi quantità di numeri** — come la media di 10.000 valori, moltiplicare matrici, o applicare funzioni statistiche — usa la libreria **NumPy**.

NumPy fa le stesse cose delle liste, ma è molto più veloce per i calcoli numerici intensivi. Lo vedremo nella sezione dedicata all'ecosistema Python.

Sintassi di base

Le regole fondamentali della sintassi Python — come "parla" il linguaggio.

Perché le regole esistono?

Ogni linguaggio, anche quelli umani come l'italiano, ha delle regole grammaticali. Se scrivi "io mangio la pizza" funziona, ma se scrivi "pizza la mangio io" il significato cambia, e in certi casi non si capisce più nulla.

Python funziona allo stesso modo: ha delle **regole di sintassi** che devi rispettare. Se le rompi, Python si ferma e ti mostra un errore. Non perché sia cattivo, ma perché non riesce a capire cosa vuoi fare.

L'indentazione: gli spazi contano!

Questa è la regola più importante e più diversa rispetto ad altri linguaggi.

In Python, gli **spazi all'inizio di una riga** non sono opzionali o decorativi: **fanno parte del codice**. Servono a dire a Python quali istruzioni stanno "dentro" ad un blocco (come un `if` o un ciclo) e quali invece stanno "fuori".

Immagina di scrivere una ricetta:

```
Se hai la farina:
    mescola con l'acqua
    aggiungi il sale
prepara il forno
```

Le righe rientrate (con gli spazi davanti) appartengono alla condizione "se hai la farina". L'ultima riga invece vale sempre, indipendentemente dalla farina. Python funziona esattamente così.

```
if 5 > 3:
    print("Cinque è maggiore di tre") # questa riga è DENTRO l'if
print("Questa riga è FUORI dall'if")  # questa viene eseguita sempre
```

Lo standard è usare **4 spazi** per ogni livello di rientro. Non mescolare spazi e tabulazioni (il tasto Tab): causa problemi.

Se dimentichi il rientro, Python ti avvisa con un errore chiarissimo:

```
if 5 > 3:
print("Errore!") # IndentationError: expected an indented block
```

Maiuscole e minuscole

Python distingue sempre tra lettere maiuscole e minuscole. Questo si chiama essere **case-sensitive**.

Quindi `nome`, `Nome` e `NOME` sono **tre cose completamente diverse** per Python:

```
nome = "Alice"
Nome = "Bob"
print(nome) # Alice
print(Nome) # Bob
```

Fai attenzione anche a `True` e `False`: devono avere la prima lettera maiuscola. `true` o `TRUE` causano un errore.

Istruzioni su più righe

Di solito scrivi un'istruzione per riga. Se un'istruzione è molto lunga e vuoi spezzarla, puoi usare il carattere `\` alla fine della riga per dire a Python "continua alla riga successiva":

```
risultato = 10 + 20 + \
            30 + 40
print(risultato) # 100
```

Oppure, se apri una parentesi, Python capisce da solo che l'istruzione non è ancora finita:

```
numeri = (1, 2, 3,
          4, 5, 6)
```

Il punto e virgola (non è necessario)

In alcuni linguaggi, ogni istruzione deve finire con `;`. In Python **non serve**. Una riga = un'istruzione, e basta.

Potresti scrivere più istruzioni sulla stessa riga usando `;`, ma è una cattiva abitudine che rende il codice difficile da leggere:

```
# Funziona, ma è meglio evitarlo
a = 1; b = 2; print(a + b)

# Molto meglio così:
a = 1
b = 2
print(a + b)
```

Le parole riservate (keywords)

Python ha alcune parole speciali che hanno già un significato preciso nel linguaggio. Non puoi usarle come nomi di variabili.

Per esempio, `if` significa "se" per Python. Se provi a creare una variabile chiamata `if`, Python si confonde e dà errore.

Queste parole si chiamano **keyword**:

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Non devi memorizzarle tutte ora. Le imparerai una alla volta quando ne avrai bisogno. L'importante è sapere che esistono: se Python ti dà un errore strano su un nome di variabile, controlla che non sia una keyword.

Booleani

I valori vero/falso in Python — la base di ogni decisione nel codice.

Cos'è un booleano?

Nella vita di tutti i giorni facciamo continuamente valutazioni del tipo "vero o falso":

- La luce è accesa? Sì o No.
- Sono maggiorenne? Vero o Falso.
- Fa freddo fuori? Vero o Falso.

In Python esistono esattamente questi due valori: **True** (vero) e **False** (falso). Il tipo di dato che li contiene si chiama **booleano** (`bool`).

Attenzione: la prima lettera è **sempre maiuscola**. `true` e `false` (tutto minuscolo) causano un errore.

```
luce_accesa = True
fa_freddo = False
print(type(luce_accesa))  # <class 'bool'>
```

I confronti producono booleani

Ogni volta che confronti due valori, Python restituisce **True** o **False** :

```
print(5 > 3)      # True  - 5 è maggiore di 3? Sì
print(5 < 3)      # False - 5 è minore di 3? No
print(5 == 5)     # True  - 5 è uguale a 5? Sì
print(5 != 3)     # True  - 5 è diverso da 3? Sì
print(5 >= 5)     # True  - 5 è maggiore o uguale a 5? Sì
print(5 <= 4)     # False - 5 è minore o uguale a 4? No
```

⚠ `=` e `==` sono cose diverse!

- `=` assegna un valore a una variabile: `x = 5`
- `==` confronta due valori: `x == 5` (è uguale a 5?) ⚠

Combinare condizioni con **and**, **or**, **not**

Spesso vuoi combinare più condizioni insieme. In Python si usano tre operatori logici.

and — "e"

Restituisce `True` solo se **tutte** le condizioni sono vere. Come il semaforo verde: puoi passare solo se la strada è libera E il semaforo è verde.

```
eta = 20
ha_patente = True

# Puoi guidare solo se sei maggiorenne E hai la patente
print(eta >= 18 and ha_patente) # True
```

or — "o"

Restituisce `True` se **almeno una** condizione è vera. Come entrare in un cinema: puoi entrare se hai il biglietto O sei un membro del club.

```
ha_biglietto = False
e_membro = True

print(ha_biglietto or e_membro) # True — basta una delle due
```

not — "non"

Inverte il valore: trasforma `True` in `False` e viceversa.

```
piove = False
print(not piove) # True — NON piove, quindi posso uscire

maggiorenne = True
print(not maggiorenne) # False
```

Valori "truthy" e "falsy"

In Python, non solo `True` e `False` possono essere trattati come booleani. **Ogni valore** può essere interpretato come vero o falso.

I valori che Python considera **falsi** (equivalenti a `False`):

- `False`
- `0` e `0.0`
- `""` (stringa vuota, cioè nessun testo)
- `[]` (lista vuota)
- `None` (assenza di valore)

Tutto il resto è considerato **vero**. Un numero diverso da zero, una stringa con del testo, una lista con elementi.

```
print(bool(0))      # False – zero è falso
print(bool(""))     # False – stringa vuota è falsa
print(bool([]))     # False – lista vuota è falsa

print(bool(42))     # True  – qualsiasi numero non-zero è vero
print(bool("ciao")) # True  – qualsiasi testo non-vuoto è vero
print(bool([1, 2])) # True  – una lista con elementi è vera
```

Questo torna molto utile nelle condizioni. Per esempio, puoi controllare se un nome è stato inserito così:

```
nome = ""
if nome:
    print("Ciao, " + nome + "!")
else:
    print("Non hai inserito nessun nome.") # questa viene stampata
```

Commenti in Python

Come lasciare note nel codice per te stesso e per chi leggerà il tuo lavoro.

Perché scrivere commenti?

Immagina di aprire un tuo vecchio quaderno di appunti dopo sei mesi. Se hai scritto solo formule e numeri senza spiegazioni, probabilmente non ricordi più cosa significano.

Nel codice succede la stessa cosa. Un **commento** è una nota che scrivi nel codice per spiegare cosa sta succedendo. Python ignora completamente i commenti quando esegue il programma: sono messaggi solo per gli esseri umani.

Scrivere buoni commenti è una delle abitudini più importanti di un programmatore.

Commenti su una riga

In Python, un commento inizia con il simbolo `#`. Tutto quello che viene scritto dopo `#` sulla stessa riga viene ignorato da Python:

```
# Questo è un commento: Python lo ignora completamente
print("Ciao!") # Puoi mettere un commento anche dopo il codice

# Puoi usare i commenti per "disattivare" del codice temporaneamente:
# print("Questa riga non viene eseguita")
```

Commenti su più righe

Se vuoi scrivere un commento lungo, hai due opzioni.

Opzione 1: Metti `#` all'inizio di ogni riga

```
# Questo programma calcola l'area di un rettangolo.
# Prende la base e l'altezza come input
# e stampa il risultato.
```

Opzione 2: Usa le virgolette triple `"""..."""`

```
"""
```

```
Questo è un blocco di testo su più righe.
```

```
Python non lo esegue come codice.
```

```
Viene spesso usato all'inizio di un file per descriverlo.
```

```
"""
```

Tecnicamente le virgolette triple creano una stringa di testo, non un vero commento. Ma se non la assegna a nessuna variabile, Python la ignora lo stesso. Vengono usate moltissimo in pratica.

Cosa scrivere (e cosa NON scrivere)

Un buon commento spiega il **perché** di una scelta, non il **cosa** fa il codice (quello si dovrebbe capire già leggendo il codice).

```
# Male: il commento dice l'ovvio, non aggiunge nulla
```

```
x = 10 # assegna 10 a x
```

```
# Bene: spiega perché usiamo quel numero specifico
```

```
VELOCITA_MASSIMA = 130 # limite autostradale in Italia, in km/h
```

```
# Bene: spiega una formula non ovvia
```

```
area = 3.14159 * raggio ** 2 # formula del cerchio: pi * r2
```

Le docstring: commenti per le funzioni

Le **docstring** sono commenti speciali che si mettono subito dentro una funzione (o una classe) per spiegare cosa fa. Si scrivono con le virgolette triple:

```
def calcola_area(base, altezza):  
    """Calcola e restituisce l'area di un rettangolo."""  
    return base * altezza  
  
def saluta(nome):  
    """  
    Stampa un messaggio di benvenuto personalizzato.  
  
    Parametri:  
    - nome: il nome della persona da salutare  
    """  
    print("Ciao, " + nome + "!")
```

Le docstring sono utili anche perché puoi leggerle dal programma stesso con la funzione `help()` :

```
help(calcola_area)  
# Stampa: Calcola e restituisce l'area di un rettangolo.
```

Tipi di dati in Python

I diversi tipi di informazione che Python sa gestire.

Perché esistono i tipi di dati?

Il computer gestisce informazioni molto diverse tra loro: numeri, testi, valori vero/falso. Non ha senso sommare del testo come se fossero numeri, o confrontare un nome con un prezzo.

Per questo ogni valore in Python ha un **tipo di dato**: una categoria che dice a Python com'è fatto quel valore e cosa puoi farci.

I tipi principali

Numero intero — `int`

Un numero senza virgola. Può essere positivo, negativo o zero.

```
eta = 16
temperatura = -5
anno = 2024
milione = 1_000_000 # puoi usare _ come separatore per leggere meglio i
grandi numeri

print(type(eta)) # <class 'int'>
```

Numero decimale — `float`

Un numero con la virgola. In Python (e nella programmazione in generale) si usa il **punto** invece della virgola.

```
altezza = 1.75
pi_greco = 3.14159
temperatura = -2.5

print(type(altezza)) # <class 'float'>
```

Testo — `str` (stringa)

Qualsiasi sequenza di caratteri: lettere, numeri, simboli. Si scrive tra virgolette singole `'...'` o doppie `"..."` :

```
nome = "Alice"
messaggio = 'Ciao, mondo!'
frase = """Questa è una stringa
su più righe."""

print(type(nome)) # <class 'str'>
```

Il termine "stringa" deriva dall'inglese *string* (filo), perché è come una sequenza di caratteri infilati uno dopo l'altro.

Vero o Falso — `bool` (booleano)

Solo due valori possibili: `True` (vero) o `False` (falso). Nota la lettera maiuscola.

```
maggiorenne = True
piove = False

print(type(maggiorenne)) # <class 'bool'>
```

Assenza di valore — `None`

`None` è un valore speciale che significa "niente". È utile per indicare che una variabile esiste ma non contiene ancora nessun dato.

```
risultato = None

print(type(risultato)) # <class 'NoneType'>
```

È come una scatola vuota: la scatola c'è, ma non c'è niente dentro.

Tipi per raccogliere più valori insieme

Oltre ai tipi semplici, Python ha tipi che contengono più valori contemporaneamente. Li vedremo in dettaglio nei capitoli successivi, ma è bene sapere che esistono:

Tipo	Esempio	Cosa è
<code>list</code> (lista)	<code>[1, 2, 3]</code>	Una sequenza ordinata di elementi, modificabile
<code>tuple</code>	<code>(1, 2, 3)</code>	Come una lista, ma non modificabile
<code>set</code> (insieme)	<code>{1, 2, 3}</code>	Una raccolta senza duplicati e senza ordine fisso
<code>dict</code> (dizionario)	<code>{"nome": "Alice"}</code>	Coppie chiave-valore, come un vocabolario

Come Python assegna il tipo automaticamente

In Python non devi dichiarare esplicitamente il tipo di una variabile: Python lo intuisce dal valore che assegna.

```
x = 10      # Python capisce che è un int
y = 3.14    # Python capisce che è un float
z = "ciao"  # Python capisce che è una str
```

Questo si chiama **tipizzazione dinamica**: il tipo non è fisso, e puoi anche cambiarlo nel corso del programma (anche se di solito non è una buona idea).

Riepilogo rapido

intero	= 10	# int	– numero senza virgola
decimale	= 3.14	# float	– numero con virgola
testo	= "Python"	# str	– testo
vero_o_falso	= True	# bool	– vero o falso
niente	= None	# None	– assenza di valore
lista	= [1, 2, 3]	# list	– sequenza modificabile
tupla	= (1, 2, 3)	# tuple	– sequenza fissa
insieme	= {1, 2, 3}	# set	– raccolta senza duplicati
dizionario	= {"a": 1}	# dict	– coppie chiave-valore

Operatori in Python

I simboli con cui fare calcoli, confronti e operazioni logiche in Python.

Cosa sono gli operatori?

Un **operatore** è un simbolo che esegue un'operazione su uno o più valori. Li usi già ogni giorno: `+`, `-`, `*` sono operatori matematici. In Python ce ne sono molti altri.

Operatori aritmetici — per i calcoli

Funzionano come la matematica che conosci già, con qualche aggiunta:

Operatore	Significato	Esempio	Risultato
<code>+</code>	Addizione	<code>5 + 3</code>	<code>8</code>
<code>-</code>	Sottrazione	<code>5 - 3</code>	<code>2</code>
<code>*</code>	Moltiplicazione	<code>5 * 3</code>	<code>15</code>
<code>/</code>	Divisione (con decimali)	<code>5 / 2</code>	<code>2.5</code>
<code>//</code>	Divisione intera (senza decimali)	<code>5 // 2</code>	<code>2</code>
<code>%</code>	Resto della divisione	<code>5 % 2</code>	<code>1</code>
<code>**</code>	Potenza	<code>5 ** 2</code>	<code>25</code>

L'operatore `//` (doppio slash) divide e butta via i decimali. È come chiedere "quante volte 2 entra in 5?" — la risposta è 2, con 1 di resto.

L'operatore `%` (modulo) restituisce solo il resto. Torna utilissimo per sapere se un numero è pari o dispari: se `numero % 2` vale `0`, il numero è pari.

```
a = 10
b = 3
print(a + b) # 13
print(a - b) # 7
print(a * b) # 30
print(a / b) # 3.3333...
print(a // b) # 3 (10 diviso 3 fa 3 con il resto)
print(a % b) # 1 (il resto è 1)
print(a ** b) # 1000 (10 alla potenza 3)
```

Operatori di confronto — per fare domande

Confrontano due valori e restituiscono sempre `True` o `False` :

Operatore	Significato	Esempio
<code>==</code>	È uguale a?	<code>5 == 5</code> → True
<code>!=</code>	È diverso da?	<code>5 != 3</code> → True
<code>></code>	È maggiore di?	<code>5 > 4</code> → True
<code><</code>	È minore di?	<code>5 < 3</code> → False
<code>>=</code>	È maggiore o uguale?	<code>5 >= 5</code> → True
<code><=</code>	È minore o uguale?	<code>5 <= 4</code> → False

```
x = 5
print(x == 5) # True
print(x != 3) # True
print(x > 4)  # True
print(x < 3)  # False
```

Operatori logici — per combinare condizioni

Servono a mettere insieme più condizioni:

Operatore	Significato
<code>and</code>	Vero solo se tutte le condizioni sono vere
<code>or</code>	Vero se almeno una condizione è vera
<code>not</code>	Inverte il valore (vero diventa falso e viceversa)

```
print(5 > 3 and 10 > 7) # True  - entrambe vere
print(5 > 3 or 10 < 7)  # True  - almeno una vera
print(not 5 > 3)         # False - 5 > 3 è True, not lo inverte
```

Operatori di assegnazione — scorciatoie per aggiornare variabili

Invece di scrivere `x = x + 3`, puoi usare la versione abbreviata `x += 3`. Significano la stessa cosa:

Abbreviazione	Equivalente completo
<code>x += 3</code>	<code>x = x + 3</code>
<code>x -= 3</code>	<code>x = x - 3</code>
<code>x *= 3</code>	<code>x = x * 3</code>
<code>x /= 3</code>	<code>x = x / 3</code>
<code>x //= 3</code>	<code>x = x // 3</code>
<code>x %= 3</code>	<code>x = x % 3</code>
<code>x **= 3</code>	<code>x = x ** 3</code>

```
punteggio = 100
punteggio += 50  # il giocatore ha guadagnato 50 punti
print(punteggio) # 150

punteggio -= 20  # penalità di 20 punti
print(punteggio) # 130
```

Operatori `in` e `not in` — per cercare elementi

Verificano se un valore è presente (o assente) in una lista, in una stringa o in qualsiasi altra sequenza:

```
frutta = ["mela", "banana", "ciliegia"]
print("mela" in frutta)      # True  - c'è la mela
print("uva" in frutta)      # False - non c'è l'uva
print("uva" not in frutta)   # True  - l'uva non c'è

testo = "Ciao, mondo!"
print("mondo" in testo)     # True
print("pizza" in testo)     # False
```

L'ordine delle operazioni

Come in matematica, Python esegue le operazioni in un ordine preciso. Le parentesi hanno sempre la precedenza massima:

```
print(2 + 3 * 4)    # 14 - prima la moltiplicazione, poi l'addizione
print((2 + 3) * 4)  # 20 - prima le parentesi
```

Se non sei sicuro dell'ordine, usa le parentesi: rendono il codice più chiaro e sicuro.

Conversione dei tipi

Come trasformare un valore da un tipo all'altro in Python.

Perché convertire i tipi?

Immagina di voler stampare questo messaggio: `"Hai 16 anni"`. Il numero `16` è un intero, ma per metterlo in mezzo al testo devi trasformarlo in una stringa di testo. Senza questa trasformazione, Python si ferma con un errore.

Questa operazione — trasformare un valore da un tipo all'altro — si chiama **conversione di tipo**.

Conversione automatica (implicita)

A volte Python fa la conversione da solo, senza che tu debba fare niente. Succede quando sommi un numero intero con uno decimale: Python trasforma automaticamente l'intero in decimale per poter fare il calcolo.

```
intero = 5
decimale = 2.5
risultato = intero + decimale

print(risultato)          # 7.5
print(type(risultato))    # <class 'float'>
```

Python fa questo solo quando è sicuro di non perdere informazioni.

Conversione manuale (esplicita)

Altre volte devi essere tu a dire a Python di convertire. Si usano funzioni apposite.

int() — trasforma in numero intero

Ecco come `int()` converte valori di tipo diverso in un numero intero:

```
x = int(3.9)      # 3 - la parte decimale viene TAGLIATA (non arrotondata!)
y = int("42")     # 42 - una stringa numerica diventa un numero
z = int(True)     # 1
w = int(False)    # 0

print(x, y, z, w) # 3 42 1 0
```

!!!caution `int(3.9)` restituisce `3`, non `4`. Il decimale viene tagliato, non arrotondato. Se vuoi arrotondare, usa `round(3.9)` che fa `4`. !!!

float() — trasforma in numero decimale

Ecco come `float()` trasforma un valore in un numero con la virgola:

```
a = float(5)      # 5.0
b = float("3.14") # 3.14
d = float(True)   # 1.0

print(a, b, d)    # 5.0 3.14 1.0
```

str() — trasforma in testo

Questa è la conversione più usata, perché spesso devi mettere numeri dentro messaggi di testo:

```
eta = 16
# print("Ho " + eta + " anni")      # ERRORE — non puoi sommare str e int
print("Ho " + str(eta) + " anni")    # OK — "Ho 16 anni"

n = str(42)      # "42"
f = str(3.14)    # "3.14"
b = str(True)    # "True"
```

In alternativa alle conversioni manuali, puoi usare le **f-string** (stringhe formattate), che gestiscono tutto automaticamente:

```
eta = 16
print(f"Ho {eta} anni") # "Ho 16 anni" — più comodo!
```

bool() — trasforma in vero/falso

Ogni valore in Python può essere convertito in booleano. I valori "vuoti" o "zero" diventano **False**, tutto il resto diventa **True** :

```
print(bool(0))      # False – zero è falso
print(bool(""))     # False – testo vuoto è falso
print(bool([]))     # False – lista vuota è falsa
print(bool(None))   # False

print(bool(1))      # True
print(bool(-5))     # True – qualsiasi numero diverso da zero è vero
print(bool("ciao")) # True – qualsiasi testo non vuoto è vero
```

Cosa succede se la conversione non è possibile?

Se provi a convertire qualcosa che non ha senso, Python ti dà un `ValueError` :

```
int("ciao")      # ValueError – "ciao" non è un numero!
float("abc")     # ValueError
```

Per gestire questo caso senza far crashare il programma, puoi usare un blocco `try/except` (che vedremo in dettaglio nel capitolo sugli errori):

```
valore_inserito = "pippo"
try:
    numero = int(valore_inserito)
    print("Numero valido:", numero)
except ValueError:
    print("Quello che hai scritto non è un numero!")
```

Variabili in Python

Come conservare e usare informazioni nel tuo programma Python.

Cos'è una variabile?

Immagina di avere una serie di scatole etichettate. In ogni scatola puoi mettere un oggetto, e sull'etichetta scrivi il nome per ritrovarla facilmente.

Una **variabile** funziona esattamente così:

- L'etichetta è il **nome** della variabile
- Il contenuto della scatola è il suo **valore**
- La scatola in sé è lo spazio in **memoria** che il computer riserva per conservare quel dato

Creare una variabile

In Python creare una variabile è semplicissimo: basta scegliere un nome e assegnarle un valore con il simbolo `=`.

```
nome = "Alice"      # una scatola chiamata "nome" con dentro il testo "Alice"
eta = 16            # una scatola chiamata "eta" con dentro il numero 16
altezza = 1.65      # una scatola chiamata "altezza" con dentro il numero 1.65
```

Non devi dire a Python che tipo di dato stai mettendo dentro: lo capisce da solo. Non appena scrivi questa riga, la variabile esiste ed è pronta all'uso.

Usare una variabile

Una volta creata, puoi usare il nome della variabile ogni volta che ti serve il suo valore:

```
nome = "Alice"
print(nome)          # Alice
print("Ciao, " + nome + "!") # Ciao, Alice!
```

Cambiare il valore

Puoi cambiare il contenuto di una variabile in qualsiasi momento. È come svuotare la scatola e metterci qualcosa di nuovo:

```
x = 10
print(x)    # 10
x = 25
print(x)    # 25
```

Puoi anche usare il vecchio valore per calcolarne uno nuovo:

```
punteggio = 100
punteggio = punteggio + 50 # oppure: punteggio += 50
print(punteggio)          # 150
```

Assegnare più variabili in una volta

Python ti permette di creare più variabili in una sola riga:

```
a, b, c = 1, 2, 3
print(a) # 1
print(b) # 2
print(c) # 3
```

Oppure puoi assegnare lo stesso valore a più variabili contemporaneamente:

```
x = y = z = 0
print(x, y, z) # 0 0 0
```

Regole per scegliere i nomi

Non puoi chiamare una variabile in qualsiasi modo. Le regole sono:

- Il nome deve iniziare con una lettera (`a-z` , `A-Z`) oppure con il trattino basso `_`
- Può contenere lettere, numeri e trattini bassi
- **Non può contenere spazi** (usa il trattino basso al loro posto)
- **Non può iniziare con un numero**

- **Non può essere una parola riservata** di Python (come `if`, `for`, `print`, ecc.)

```
# Nomi validi ✓
nome = "Alice"
eta_utente = 16
_privato = 42
valore2 = 3.14

# Nomi non validi ✗
# 2variabile = 10    - inizia con un numero
# mio nome = "Bob"   - contiene uno spazio
# if = "A"           - è una parola riservata di Python
```

Lo stile: snake_case

Per convenzione, i nomi delle variabili in Python si scrivono in **snake_case**: tutto minuscolo, con il trattino basso `_` tra le parole, come una serpe (snake) strisciante.

```
# Stile consigliato ✓
nome_utente = "Alice"
eta_minima = 18
numero_di_studenti = 30

# Stile sconsigliato (funziona, ma non è la norma)
NomeUtente = "Alice"    # CamelCase, si usa per le classi
ETAMINIMA = 18           # tutto maiuscolo, si usa per le costanti
```

Scoprire il tipo di una variabile

Puoi chiedere a Python che tipo di dato contiene una variabile con la funzione `type()` :

```
x = 42
print(type(x))      # <class 'int'> - numero intero

y = "ciao"
print(type(y))      # <class 'str'> - stringa di testo

z = 3.14
print(type(z))      # <class 'float'> - numero decimale
```

Questo torna utile quando non sei sicuro di cosa contiene una variabile.

Dizionari

Come associare informazioni tramite chiavi e valori, come un vero vocabolario.

Cosa sono i dizionari?

Un **dizionario** è una raccolta di informazioni organizzate in coppie **chiave → valore**.

Funziona come un vero vocabolario: cerchi una parola (la **chiave**) e trovi la sua definizione (il **valore**). Oppure come una rubrica telefonica: cerchi il nome di una persona e trovi il suo numero.

```
"nome" → "Alice"
"eta" → 16
"citta" → "Roma"
```

Creare un dizionario

I dizionari si scrivono con le **parentesi graffe** `{ }`, con coppie `"chiave": valore` separate da virgole:

```
studente = {  
    "nome": "Alice",  
    "eta": 16,  
    "classe": "3A"  
}  
  
vuoto = {}    # dizionario senza elementi
```

Accedere a un valore

Usi il nome della chiave tra parentesi quadre, come se fosse un indice:

```
studente = {"nome": "Alice", "eta": 16, "classe": "3A"}  
  
print(studente["nome"])    # Alice  
print(studente["eta"])     # 16
```

Se la chiave non esiste, ottieni un errore. Per evitarlo, usa `.get()`, che restituisce `None` (o un valore di default) invece di causare errore:

```
print(studente.get("cognome"))    # None – nessun errore  
print(studente.get("cognome", "N/D"))    # N/D – valore di default
```

Aggiungere e modificare

I dizionari sono modificabili. Assegna un valore a una chiave: se la chiave esiste, modifica il valore; se non esiste, crea una nuova coppia:

```
studente = {"nome": "Alice", "eta": 16}

studente["eta"] = 17          # modifica il valore esistente
studente["cognome"] = "Rossi" # aggiunge una nuova coppia

print(studente)
# {'nome': 'Alice', 'eta': 17, 'cognome': 'Rossi'}
```

Rimuovere elementi

```
studente = {"nome": "Alice", "eta": 16, "classe": "3A"}

del studente["classe"]      # elimina la coppia "classe"
print(studente) # {'nome': 'Alice', 'eta': 16}

eta = studente.pop("eta")   # rimuove e restituisce il valore
print(eta)                 # 16
print(studente) # {'nome': 'Alice'}
```

Strumenti utili

```
persona = {"nome": "Alice", "eta": 16, "citta": "Roma"}

# Tutte le chiavi
print(persona.keys())    # dict_keys(['nome', 'eta', 'citta'])

# Tutti i valori
print(persona.values())  # dict_values(['Alice', 16, 'Roma'])

# Tutte le coppie chiave-valore
print(persona.items())   # dict_items([('nome', 'Alice'), ('eta', 16), ('citta', 'Roma')])

# Verificare se una chiave esiste
print("nome" in persona)    # True
print("cognome" in persona) # False
```

Scorrere un dizionario

```
persona = {"nome": "Alice", "eta": 16, "citta": "Roma"}

# Solo le chiavi
for chiave in persona:
    print(chiave)    # nome, eta, citta

# Chiavi e valori insieme
for chiave, valore in persona.items():
    print(f"{chiave}: {valore}")
```

Output:

```
nome: Alice
eta: 16
citta: Roma
```

Dizionari dentro dizionari

I valori possono essere qualsiasi cosa, anche altri dizionari. Utile per rappresentare strutture complesse:

```
classe = {  
    "Alice": {"eta": 16, "voto": 8},  
    "Bob":   {"eta": 17, "voto": 7}  
}  
  
print(classe["Alice"]["voto"]) # 8  
print(classe["Bob"]["eta"])    # 17
```

Liste

Come raccogliere più valori in una sequenza ordinata e modificabile.

Cosa sono le liste?

Immagina un foglio con una lista della spesa: ci scrivi sopra gli elementi uno sotto l'altro, puoi aggiungerne di nuovi, cancellarne, cambiarli. In Python le **liste** funzionano esattamente così.

Una lista è una raccolta ordinata di elementi. Puoi metterci qualsiasi tipo di valore, anche misti, e puoi modificarla liberamente dopo averla creata.

Creare una lista

Le liste si scrivono con le **parentesi quadre** `[]`, con gli elementi separati da virgole:

```
frutta = ["mela", "banana", "ciliegia"]  
numeri = [1, 2, 3, 4, 5]  
mista = [1, "ciao", True, 3.14] # anche tipi diversi vanno bene  
vuota = []                      # lista senza elementi
```

Accedere agli elementi

Ogni elemento ha una **posizione** (chiamata indice) che parte da **0**, non da **1**. È una convenzione tipica della programmazione.

```
frutta = ["mela", "banana", "ciliegia"]
print(frutta[0])    # mela      - il primo elemento
print(frutta[1])    # banana    - il secondo
print(frutta[-1])   # ciliegia  - l'ultimo (conta dalla fine)
print(frutta[-2])   # banana    - il penultimo
```

Estrarre una parte della lista (slicing)

Puoi estrarre una "fetta" della lista indicando da dove a dove:

```
numeri = [0, 1, 2, 3, 4, 5]
print(numeri[1:4])  # [1, 2, 3]  - dalla posizione 1 alla 3 (4 esclusa)
print(numeri[:3])   # [0, 1, 2]  - dall'inizio alla posizione 2
print(numeri[3:])   # [3, 4, 5]  - dalla posizione 3 alla fine
print(numeri[::-1]) # [5, 4, 3, 2, 1, 0] - la lista al contrario
```

Modificare gli elementi

A differenza delle tuple (che vedremo dopo), le liste **si possono modificare**:

```
frutta = ["mela", "banana", "ciliegia"]
frutta[1] = "kiwi"    # sostituisce "banana" con "kiwi"
print(frutta)         # ['mela', 'kiwi', 'ciliegia']
```

Aggiungere elementi

Python ti dà tre modi per aggiungere elementi a una lista, a seconda di dove vuoi inserirli:

```

frutta = ["mela", "banana"]

frutta.append("ciliegia")      # aggiunge alla fine
print(frutta) # ['mela', 'banana', 'ciliegia']

frutta.insert(1, "kiwi")      # inserisce nella posizione 1
print(frutta) # ['mela', 'kiwi', 'banana', 'ciliegia']

altra = ["uva", "pesca"]
frutta.extend(altra)          # aggiunge tutti gli elementi di un'altra
                                lista
print(frutta) # ['mela', 'kiwi', 'banana', 'ciliegia', 'uva', 'pesca']

```

Rimuovere elementi

Anche per rimuovere hai più opzioni, a seconda di cosa conosci: il valore o la posizione:

```

frutta = ["mela", "banana", "ciliegia", "banana"]

frutta.remove("banana")      # rimuove la prima "banana" che trova
print(frutta) # ['mela', 'ciliegia', 'banana']

frutta.pop()                 # rimuove e restituisce l'ultimo elemento
print(frutta) # ['mela', 'ciliegia']

frutta = ["mela", "banana", "ciliegia"]
del frutta[1]                # elimina l'elemento in posizione 1
print(frutta) # ['mela', 'ciliegia']

frutta.clear()               # svuota completamente la lista
print(frutta) # []

```

Cercare e contare

```
numeri = [1, 2, 3, 2, 4, 2]
print(numeri.count(2))    # 3 - quante volte compare il numero 2
print(numeri.index(3))    # 2 - in quale posizione si trova il numero 3
print(len(numeri))        # 6 - quanti elementi ha la lista
```

Ordinare una lista

```
numeri = [3, 1, 4, 1, 5, 9, 2]

numeri.sort()              # ordina la lista (modifica l'originale)
print(numeri) # [1, 1, 2, 3, 4, 5, 9]

numeri.sort(reverse=True)  # ordine decrescente
print(numeri) # [9, 5, 4, 3, 2, 1, 1]

# sorted() non modifica l'originale, restituisce una nuova lista
originale = [3, 1, 4, 1, 5]
ordinata = sorted(originale)
print(originale) # [3, 1, 4, 1, 5] - invariata
print(ordinata)  # [1, 1, 3, 4, 5] - nuova lista ordinata
```

Copiare una lista

Attenzione: se scrivi `copia = originale`, non stai copiando la lista, stai solo creando un secondo nome per la stessa lista! Usa `.copy()`:

```
originale = [1, 2, 3]
copia = originale.copy()    # copia vera e propria
copia.append(4)

print(originale) # [1, 2, 3] - non cambiata
print(copia)     # [1, 2, 3, 4] - modificata
```

Liste di liste

Puoi mettere liste dentro altre liste, utile per rappresentare griglie o tabelle:

```
matrice = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
print(matrice[1][2]) # 6 - seconda riga, terza colonna
```

Set (Insiemi)

Raccolte di elementi unici — perfetti per eliminare i duplicati e fare operazioni sugli insiemi.

Cosa sono i set?

Un **set** (insieme) è una raccolta di elementi con due regole fondamentali:

1. **Nessun duplicato**: ogni elemento può comparire al massimo una volta
2. **Nessun ordine garantito**: gli elementi non hanno una posizione fissa

Pensa a un set come a un sacchetto di palline colorate: sai quali colori ci sono dentro, ma non puoi dire "dammi la terza pallina".

Creare un set

Si usano le **parentesi graffe** `{}`, come per i dizionari, ma senza le chiavi:

```
frutta = {"mela", "banana", "ciliegia"}  
numeri = {1, 2, 3, 4, 5}
```

⚠ `{}` da solo crea un **dizionario vuoto**, non un set vuoto! Per creare un set vuoto usa `set()`:

```
vuoto = set()      # set vuoto ✓  
anche_vuoto = {}  # dizionario vuoto, non un set! ✗
```

...

I duplicati vengono rimossi automaticamente:

```
con_duplicati = {1, 2, 2, 3, 3, 3}  
print(con_duplicati)  # {1, 2, 3} - solo valori unici
```

Aggiungere e rimuovere elementi

```
frutta = {"mela", "banana"}  
  
frutta.add("ciliegia")    # aggiunge un elemento  
frutta.remove("banana")  # rimuove (dà errore se non esiste)  
frutta.discard("uva")     # rimuove senza errore se non esiste
```

Controllare se un elemento è presente

```
frutta = {"mela", "banana", "ciliegia"}  
print("mela" in frutta)    # True  
print("uva" in frutta)     # False
```

I set sono molto veloci per questo tipo di ricerca, anche più delle liste.

Operazioni sugli insiemi

Qui i set diventano particolarmente utili: puoi fare operazioni tipiche della matematica degli insiemi.

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
```

Unione — tutti gli elementi di A e di B:

```
print(A | B)  # {1, 2, 3, 4, 5, 6}
```

Intersezione — solo gli elementi presenti in entrambi:

```
print(A & B)  # {3, 4}
```

Differenza — gli elementi in A ma non in B:

```
print(A - B)  # {1, 2}
```

Il trucco più utile: eliminare i duplicati

Il modo più semplice per eliminare i duplicati da una lista è convertirla in set e poi riconvertirla in lista:

```
lista = [1, 2, 2, 3, 3, 4, 1]
lista_pulita = list(set(lista))
print(lista_pulita)  # [1, 2, 3, 4]
```

:::note L'ordine originale non viene preservato, perché i set non hanno un ordine fisso. :::

Limiti dei set

- Non puoi accedere agli elementi per posizione: `frutta[0]` causa errore
- Gli elementi devono essere immutabili: puoi mettere numeri, stringhe, tuple, ma non liste

Stringhe in Python

Come lavorare con il testo in Python.

Cosa sono le stringhe?

Una **stringa** è semplicemente del testo: un nome, un messaggio, una frase. In Python le stringhe si scrivono tra virgolette singole `'...'` o doppie `"..."` :

```
nome = "Alice"  
saluto = 'Ciao, mondo!'
```

Usa virgolette doppie se dentro il testo c'è un apostrofo, e singole se c'è già una virgoletta doppia:

```
con_apice = "It's a great day"      # apostrofo dentro → virgolette doppie  
fuori  
con_virgolette = 'Disse "ciao!"'    # virgolette doppie dentro → singole  
fuori
```

Per testo su più righe, usa le virgolette triple:

```
poesia = """Questa stringa  
si estende su  
più righe."""
```

Accedere ai singoli caratteri

Come nelle liste, ogni carattere ha una posizione (indice) che parte da `0` :

```
testo = "Python"
print(testo[0])    # P  - primo carattere
print(testo[1])    # y
print(testo[-1])   # n  - ultimo carattere
print(testo[-2])   # o  - penultimo
```

Estrarre una parte di stringa (slicing)

Puoi estrarre una sottostringa indicando da dove a dove:

```
testo = "Python"
print(testo[0:3])   # Pyt  - dalla posizione 0 alla 2 (3 esclusa)
print(testo[2:])    # thon - dalla posizione 2 alla fine
print(testo[:4])    # Pyth - dall'inizio alla posizione 3
print(testo[::-1])  # nohtyP - la stringa al contrario
```

Lunghezza di una stringa

```
testo = "Python"
print(len(testo))  # 6
```

Mettere stringhe insieme

Puoi unire due stringhe con `+` e ripetere una stringa con `*` :

```
a = "Ciao"
b = " mondo"
print(a + b)    # Ciao mondo
print(a * 3)    # CiaoCiaoCiao
```

Le f-string: il modo migliore per inserire variabili

Invece di unire le stringhe con `+` (che è scomodo), usa le **f-string**: scrivi una `f` prima delle virgolette e metti le variabili tra parentesi graffe `{}` :

```
nome = "Alice"
eta = 16
print(f"Mi chiamo {nome} e ho {eta} anni.")
# Mi chiamo Alice e ho 16 anni.

a = 5
b = 3
print(f"La somma di {a} e {b} è {a + b}.")
# La somma di 5 e 3 è 8.
```

Le f-string sono la forma moderna e più leggibile. Usale sempre quando devi mettere variabili nel testo.

Metodi utili delle stringhe

Le stringhe hanno molti strumenti integrati. Ecco i più usati, divisi per categoria.

Cambiare maiuscole e rimuovere spazi

```
testo = "  Ciao, Mondo!  "

print(testo.upper()) # "  CIAO, MONDO!  " - tutto maiuscolo
print(testo.lower()) # "  ciao, mondo!  " - tutto minuscolo
print(testo.strip()) # "Ciao, Mondo!" - rimuove spazi iniziali e finali
```

Cercare, sostituire e dividere

```
testo = "Ciao, Mondo!"

print(testo.replace("Mondo", "Python")) # "Ciao, Python!"

frutta = "mela,banana,ciliegia"
print(frutta.split(",")) # ['mela', 'banana', 'ciliegia'] - divide in lista
print(",".join(["mela", "banana", "ciliegia"])) # "mela,banana,ciliegia"
- unisce
```

Controllare il contenuto

```
testo = "ciao"

print(testo.capitalize()) # "Ciao" - prima lettera maiuscola
print(testo.startswith("ci")) # True - inizia con "ci"?
print(testo.endswith("ao")) # True - finisce con "ao"?
print(testo.find("ia")) # 1 - dove inizia "ia"? (-1 se non trovato)
print(testo.count("a")) # 1 - quante volte compare "a"?
```

Caratteri speciali

Alcune combinazioni di caratteri con `\` hanno un significato speciale:

Sequenza	Significato	Esempio
<code>\n</code>	Nuova riga	<code>"Prima riga\nSeconda riga"</code>
<code>\t</code>	Tabulazione (spazi)	<code>"Nome:\tAlice"</code>
<code>\\</code>	Un singolo backslash	<code>"C:\\Users\\Alice"</code>
<code>\"</code>	Virgolette doppie dentro la stringa	<code>"Disse \"ciao!\""</code>

```
print("Prima riga\nSeconda riga")
# Prima riga
# Seconda riga

print("Nome:\tAlice")
# Nome: Alice
```

Le stringhe non si modificano

Attenzione: le stringhe sono **immutabili**. Non puoi cambiare un singolo carattere. Se hai bisogno di una stringa modificata, devi crearne una nuova:

```
testo = "ciao"
# testo[0] = "C" - ERRORE! Non si può modificare

# Crea invece una nuova stringa
testo = "C" + testo[1:]
print(testo) # Ciao
```

Tuple

Sequenze di elementi che non cambiano mai.

Cosa sono le tuple?

Una **tupla** è simile a una lista: contiene elementi in ordine. La differenza fondamentale è che le tuple sono **immutabili**: una volta create, non si possono modificare. Non puoi aggiungere, togliere o cambiare gli elementi.

Pensa alle tuple come a dati "fissi": le coordinate GPS di un posto, i giorni della settimana, i colori del semaforo. Sono informazioni che non devono cambiare.

Creare una tupla

Le tuple si scrivono con le **parentesi tonde** `()` :

```
coordinate = (10, 20)
colori = ("rosso", "verde", "blu")
mista = (1, "ciao", True, 3.14)
vuota = ()
```

:::caution Una tupla con un solo elemento richiede una virgola finale, altrimenti Python la interpreta come una normale espressione tra parentesi:

```
non_e_tupla = (42)      # è solo il numero 42 tra parentesi!
e_una_tupla = (42,)    # la virgola la rende una tupla
```

:::

Accedere agli elementi

Funziona esattamente come per le liste, con gli indici:

```
colori = ("rosso", "verde", "blu")
print(colori[0])    # rosso
print(colori[-1])   # blu      - l'ultimo elemento
print(colori[1:3])  # ('verde', 'blu') - slicing
```

Non si possono modificare

Se provi a cambiare un elemento, Python dà errore:

```
colori = ("rosso", "verde", "blu")
colori[0] = "giallo" # TypeError: 'tuple' object does not support item
assignment
```

Questo è il comportamento voluto: garantisce che i dati rimangano intatti.

Operazioni sulle tuple

Anche se non si modificano, puoi comunque combinarle e controllarle:

```
a = (1, 2, 3)
b = (4, 5, 6)

print(a + b)    # (1, 2, 3, 4, 5, 6) – crea una nuova tupla unendo le due
print(a * 2)    # (1, 2, 3, 1, 2, 3) – ripete
print(3 in a)   # True – controlla se 3 è presente
print(len(a))  # 3 – numero di elementi
```

Spacchettare una tupla (unpacking)

Puoi assegnare gli elementi di una tupla a più variabili in una sola riga. Si chiama **unpacking** (spacchettamento):

```
coordinate = (10, 20)
x, y = coordinate
print(x)    # 10
print(y)    # 20

nome, eta, citta = ("Alice", 16, "Roma")
print(nome)  # Alice
print(eta)   # 16
print(citta) # Roma
```

Funziona anche con le liste. L'importante è che il numero di variabili corrisponda al numero di elementi.

Convertire tra tuple e liste

Se hai una tupla e hai bisogno di modificarla, convertila in lista, modificala e riconvertila:

```
t = (1, 2, 3)
l = list(t)      # converte in lista
l.append(4)      # modifica la lista
t = tuple(l)     # riconverte in tupla

print(t) # (1, 2, 3, 4)
```

Quando usare una tupla invece di una lista?

Usa una **tupla** quando:

- I dati non devono mai cambiare (coordinate, giorni della settimana, codici fissi)
- Vuoi comunicare a chi legge il codice che quei dati sono "di sola lettura"
- Hai bisogno di usarla come chiave di un dizionario (le liste non si possono usare come chiavi)

Usa una **lista** per tutto il resto, quando i dati possono cambiare.

Librerie esterne

Introduzione all'ecosistema di librerie Python.

Perché ti servono le librerie esterne

Python da solo è già potente, ma la sua vera forza è la comunità: milioni di programmatori di tutto il mondo hanno creato librerie che risolvono problemi comuni. Invece di scrivere tutto da zero, puoi usare il loro lavoro.

Vuoi creare un sito web? C'è una libreria. Fare intelligenza artificiale? C'è una libreria. Analizzare dati come in Excel? C'è una libreria. Sviluppare un videogioco? C'è una libreria.

Come usare una libreria esterna

Il processo è sempre lo stesso in tre passi:

Passo 1: Installa la libreria (una volta sola, dal terminale)

```
pip install requests
```

Passo 2: Importa la libreria nel tuo codice

Passo 3: Usa le funzionalità che offre

```
risposta = requests.get("https://api.github.com")
print(risposta.status_code)    # 200 = tutto ok
```

Le categorie principali

Web e internet

- **requests** — scarica dati da internet, interagisce con API web. La libreria Python più scaricata in assoluto.
- **flask** — crea siti web e API con pochissimo codice. Ottimo per imparare.
- **django** — framework web completo, usato da Instagram e Pinterest.
- **fastapi** — API moderne e velocissime, con documentazione automatica.

Dati e scienza

- **numpy** — calcoli matematici su grandi quantità di numeri, molto più veloce delle liste normali.
- **pandas** — analisi di dati tabellari (come Excel, ma programmabile e automatizzabile).
- **matplotlib** — crea grafici: istogrammi, torte, linee, dispersione, ecc.
- **scikit-learn** — algoritmi di machine learning pronti da usare.
- **tensorflow** / **pytorch** — deep learning e reti neurali.

Automazione

- **selenium** — controlla un browser in modo automatico (utile per testare siti web o automatizzare task ripetitivi).
 - **beautifulsoup4** — legge e analizza il codice HTML di pagine web per estrarne i dati.
 - **pyautogui** — muove il mouse e preme tasti in modo automatico.
 - **schedule** — esegue del codice automaticamente a orari programmati.
-

Grafica e giochi

- **pygame** — sviluppa videogiochi 2D in Python.
 - **pillow** — apri, modifica e salva immagini (ridimensiona, ritaglia, applica filtri, ecc.).
 - **tkinter** — crea finestre con pulsanti, caselle di testo e menu (già incluso in Python).
 - **PyQt** / **PySide** — interfacce grafiche più avanzate e professionali.
-

Utilità

- **tqdm** — aggiunge una barra di avanzamento ai tuoi cicli, così vedi quanto manca.
 - **rich** — stampa testo colorato e tabelle belle nel terminale.
 - **loguru** — sistema di log semplice e con output colorato.
 - **python-dotenv** — legge variabili di configurazione da un file `.env`.
-

Esempio: scaricare dati dal web con **requests**

```
pip install requests
```

```
# Chiede dati a un'API pubblica di prova
risposta = requests.get("https://jsonplaceholder.typicode.com/posts/1")

if risposta.status_code == 200:    # 200 = successo
    dati = risposta.json()        # Converte la risposta JSON in un
    dizionario Python
    print(f"Titolo: {dati['title']}")
    print(f"Testo: {dati['body'][:50]}...") # Solo i primi 50 caratteri
else:
    print(f"Errore: {risposta.status_code}")
```

Esempio: barra di avanzamento con **tqdm**

```
pip install tqdm
```

```
from tqdm import tqdm

# Aggiunge una barra di avanzamento a qualsiasi ciclo for
for i in tqdm(range(100), desc="Elaborazione"):
    time.sleep(0.05) # Simula un'operazione che richiede tempo
```

Output nel terminale:

```
Elaborazione: 100%|██████████| 100/100 [00:05<00:00, 20.00it/s]
```

Come trovare la libreria giusta

1. Controlla prima se esiste qualcosa nella **libreria standard** di Python (senza installare niente)
2. Cerca su Google: "python libreria per [quello che ti serve]"

3. Guarda le stelle su GitHub — più stelle = più popolare e affidabile
4. Controlla la data dell'ultimo aggiornamento — una libreria aggiornata di recente è un buon segno
5. Leggi brevemente la documentazione ufficiale

Il consiglio fondamentale: **non reinventare la ruota**. Se qualcosa è già stato fatto da qualcun altro in modo professionale, usalo.

NumPy

Introduzione alla libreria NumPy per il calcolo numerico in Python.

Perché è utile usare NumPy

Le liste Python vanno bene per la maggior parte dei casi. Ma quando hai bisogno di fare **calcoli matematici su migliaia o milioni di numeri**, le liste diventano lente. NumPy risolve questo problema: fa le stesse operazioni ma in modo molto più veloce.

NumPy è alla base di quasi tutto l'ecosistema scientifico di Python: pandas, matplotlib, scikit-learn usano tutti NumPy internamente.

Installazione e importazione

```
pip install numpy
```

Per convenzione, NumPy si importa sempre con il soprannome `np` :

Gli array di NumPy

La struttura centrale di NumPy è l'**array** (`ndarray`) — simile a una lista Python, ma pensata per i calcoli numerici.

```
# Crea un array da una lista Python
a = np.array([1, 2, 3, 4, 5])
print(a)          # [1 2 3 4 5]
print(type(a))    # <class 'numpy.ndarray'>
print(a.dtype)    # int64 — il tipo dei dati (tutti interi a 64 bit)
print(a.shape)    # (5,) — la forma: 5 elementi in una dimensione

# Crea una matrice (array a due dimensioni)
m = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])
print(m)
print(m.shape)    # (3, 3) — 3 righe, 3 colonne
```

Creare array già pronti

```
print(np.zeros(5))          # [0. 0. 0. 0. 0.] — cinque zeri
print(np.ones(4))           # [1. 1. 1. 1.] — quattro uni
print(np.zeros((2, 3)))     # matrice 2×3 di zeri
print(np.eye(3))            # matrice identità 3×3 (uno sulla diagonale)
print(np.arange(0, 10, 2))  # [0 2 4 6 8] — come range(), ma per array
print(np.linspace(0, 1, 5)) # [0. 0.25 0.5 0.75 1.] — 5 punti equidistanti
                             # tra 0 e 1
print(np.random.rand(3))    # 3 numeri decimali casuali tra 0 e 1
```

La potenza delle operazioni vettoriali

Con le liste Python, per moltiplicare ogni elemento per 2 dovresti scrivere un ciclo. Con NumPy, le operazioni si applicano **all'intero array in una volta sola**:

```
a = np.array([1, 2, 3, 4, 5])

# Operazioni su tutti gli elementi – senza scrivere cicli!
print(a + 10)    # [11 12 13 14 15]
print(a * 2)     # [ 2  4  6  8 10]
print(a ** 2)    # [ 1  4  9 16 25]
print(a > 3)     # [False False False  True  True]

# Operazioni tra due array (elemento per elemento)
b = np.array([10, 20, 30, 40, 50])
print(a + b)     # [11 22 33 44 55]
print(a * b)     # [ 10  40  90 160 250]
```

Questo approccio è detto **vettorializzazione**: invece di fare le operazioni una alla volta con un ciclo, NumPy le fa tutte insieme in modo ottimizzato.

Funzioni matematiche integrate

```
a = np.array([1, 4, 9, 16, 25])

print(np.sqrt(a))    # [1.  2.  3.  4.  5.] – radice quadrata di ogni elemento
print(np.sum(a))     # 55    – somma di tutti gli elementi
print(np.mean(a))    # 11.0  – media
print(np.min(a))     # 1     – valore minimo
print(np.max(a))     # 25    – valore massimo
print(np.std(a))     # deviazione standard
```

Accedere agli elementi

```
m = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

print(m[0])          # [1 2 3]          ← prima riga intera
print(m[1, 2])        # 6                ← riga 1, colonna 2
print(m[:, 0])        # [1 4 7]         ← tutta la colonna 0
print(m[0:2, 1:3])    # [[2 3] [5 6]]   ← un sotto-blocco (righe 0-1, colonne 1-2)
```

NumPy vs liste Python: la differenza di velocità

```
n = 1_000_000    # Un milione di numeri

# Metodo 1: lista Python con ciclo
lista = list(range(n))
inizio = time.time()
risultato = [x * 2 for x in lista]
print(f"Lista Python: {time.time() - inizio:.3f} secondi")

# Metodo 2: array NumPy
array = np.arange(n)
inizio = time.time()
risultato = array * 2
print(f"NumPy: {time.time() - inizio:.3f} secondi")
```

NumPy è tipicamente **10-100 volte più veloce** delle liste Python per operazioni numeriche. Su un milione di elementi, la differenza è notevole.

Quando usare NumPy vs liste

Situazione	Usa...
Dati misti (stringhe, numeri, oggetti)	Lista Python
Calcoli matematici su molti numeri	NumPy
Semplice raccolta di elementi	Lista Python
Matrici, algebra lineare, statistiche	NumPy
Qualsiasi libreria scientifica (pandas, matplotlib)	NumPy (lo usano internamente)

Pandas

Introduzione alla libreria Pandas per l'analisi dei dati in Python.

Perché è utile usare Pandas

Immagina di avere un foglio Excel con migliaia di righe e di dover trovare tutti gli studenti con media superiore all'8, ordinarli per cognome, e salvare il risultato in un nuovo file. A mano richiederebbe ore.

Con **Pandas** lo fai in poche righe di codice Python — e puoi ripeterlo automaticamente ogni volta che i dati cambiano.

Cos'è Pandas?

Pandas è la libreria Python più usata per lavorare con **dati tabellari** — cioè dati organizzati in righe e colonne, come un foglio Excel o una tabella di database.

Le due strutture principali sono:

- **Series**: una singola colonna di dati (come una lista con etichette)
- **DataFrame**: una tabella completa con righe e colonne

Installazione e importazione

```
pip install pandas
```

Per convenzione, Pandas si importa con il soprannome `pd` :

Series: una colonna di dati

```
# Crea una Series da una lista
voti = pd.Series([8, 7, 9, 6, 10])
print(voti)
# 0      8
# 1      7
# 2      9
# 3      6
# 4     10
# dtype: int64

# Crea una Series con etichette personalizzate (invece dei numeri 0, 1, 2...)
voti_classe = pd.Series(
    [8, 7, 9, 6],
    index=["Alice", "Bob", "Carlo", "Diana"]
)
print(voti_classe["Alice"])    # 8
print(voti_classe.mean())     # 7.5 - media
```

DataFrame: la tabella di dati

Un **DataFrame** è come un foglio Excel: ha righe e colonne con nomi.

```
# Crea un DataFrame da un dizionario (ogni chiave è una colonna)
studenti = pd.DataFrame({
    "nome":  ["Alice", "Bob", "Carlo", "Diana"],
    "eta":   [16, 15, 17, 16],
    "voto":  [8, 7, 9, 6],
    "citta": ["Roma", "Milano", "Roma", "Napoli"]
})

print(studenti)
```

Output:

	nome	eta	voto	citta
0	Alice	16	8	Roma
1	Bob	15	7	Milano
2	Carlo	17	9	Roma
3	Diana	16	6	Napoli

Accedere ai dati

```
# Selezionare una colonna (come accedere a un dizionario)
print(studenti["nome"])

# Selezionare più colonne
print(studenti[["nome", "voto"]])

# Selezionare righe per numero (iloc = integer location)
print(studenti.iloc[0])      # Prima riga (indice 0)
print(studenti.iloc[0:2])    # Righe 0 e 1

# Selezionare un singolo valore (riga, colonna)
print(studenti.iloc[0, 2])    # Riga 0, colonna 2 → 8 (il voto di Alice)
```

Filtrare i dati

Qui Pandas diventa potente: puoi selezionare solo le righe che soddisfano una condizione.

```
# Tutti gli studenti con voto maggiore o uguale a 8
promossi = studenti[studenti["voto"] >= 8]
print(promossi)

# Tutti gli studenti di Roma
romani = studenti[studenti["citta"] == "Roma"]
print(romani)

# Filtri combinati: voto >= 7 E età >= 16
risultato = studenti[(studenti["voto"] >= 7) & (studenti["eta"] >= 16)]
print(risultato)
```

Statistiche in un colpo solo

```
print(studenti["voto"].mean())    # 7.5  - media dei voti
print(studenti["voto"].max())     # 9    - voto più alto
print(studenti["voto"].min())     # 6    - voto più basso
print(studenti["voto"].std())     # deviazione standard

# Riepilogo statistico di tutte le colonne numeriche
print(studenti.describe())
```

Leggere e scrivere file

Una delle funzionalità più utili: Pandas legge e scrive file CSV ed Excel direttamente.

```
# Leggere un file CSV (valori separati da virgole)
df = pd.read_csv("dati.csv")

# Scrivere su CSV (index=False evita di scrivere i numeri di riga)
studenti.to_csv("studenti_export.csv", index=False)

# Leggere un file Excel
df_excel = pd.read_excel("dati.xlsx")

# Scrivere su Excel
studenti.to_excel("studenti_export.xlsx", index=False)
```

Operazioni comuni

```
# Aggiungere una nuova colonna calcolata
studenti["promosso"] = studenti["voto"] >= 6 # True o False per ogni riga
print(studenti)

# Ordinare per voto (dal più alto al più basso)
ordinati = studenti.sort_values("voto", ascending=False)
print(ordinati)

# Raggruppare e calcolare statistiche per gruppo
# (media dei voti per ogni città)
per_citta = studenti.groupby("citta")["voto"].mean()
print(per_citta)

# Informazioni generali sul DataFrame
print(studenti.shape) # (4, 4) - 4 righe, 4 colonne
print(studenti.dtypes) # tipo di dati in ogni colonna
```

Uso di pacchetti (pip)

Come installare e gestire pacchetti Python con pip.

Qual è la funzione di pip

Python ha già molte funzionalità integrate, ma la sua vera forza è l'enorme numero di **librerie esterne** create dalla comunità: strumenti per fare grafica, analisi dati, intelligenza artificiale, giochi, siti web e molto altro.

pip è lo strumento che ti permette di scaricare e installare queste librerie in pochi secondi.

Cos'è pip?

pip è il "gestore di pacchetti" ufficiale di Python. È come un negozio online: ti permette di cercare e installare gratuitamente oltre 400.000 librerie da **PyPI** (il deposito ufficiale di Python, all'indirizzo pypi.org).

Pip è già incluso con Python — non devi installarlo separatamente.

Verificare che pip funzioni

Apri il terminale (o il prompt dei comandi su Windows) e digita:

```
pip --version
# pip 24.0 from /usr/lib/python3/dist-packages/pip (python 3.x)
```

Se hai sia Python 2 che Python 3, potrebbe servire **pip3** :

```
pip3 --version
```

Installare una libreria

```
pip install nome-libreria
```

Esempio: installare `requests` , la libreria più usata per fare richieste al web:

```
pip install requests
```

Installare una versione specifica (utile quando un progetto richiede esattamente una certa versione):

```
pip install requests==2.31.0
```

Aggiornare una libreria all'ultima versione disponibile:

```
pip install --upgrade requests
```

Disinstallare una libreria

```
pip uninstall requests
```

Vedere le librerie installate

```
pip list
```

Output di esempio:

Package	Version
-----	-----
pip	24.0
requests	2.31.0
numpy	1.26.0

Il file `requirements.txt` : condividere le dipendenze

Quando condividi un progetto con altri, devono sapere quali librerie installare. Il file `requirements.txt` elenca tutto quello che serve.

Creare il file (salva le librerie attualmente installate):

```
pip freeze > requirements.txt
```

Contenuto tipico di `requirements.txt` :

```
requests==2.31.0  
numpy==1.26.0  
pandas==2.1.0
```

Installare tutto da un `requirements.txt` (utile quando ricevi un progetto da qualcun altro):

```
pip install -r requirements.txt
```

Ottenere informazioni su una libreria

```
pip show requests
```

Output:

```
Name: requests
Version: 2.31.0
Summary: Python HTTP for Humans.
Home-page: https://requests.readthedocs.io
```

Librerie popolari da conoscere

Libreria	A cosa serve
<code>requests</code>	Scaricare dati dal web, fare richieste HTTP
<code>numpy</code>	Calcoli matematici veloci su grandi quantità di numeri
<code>pandas</code>	Analisi e manipolazione di dati (come Excel ma programmabile)
<code>matplotlib</code>	Creare grafici e visualizzazioni
<code>flask</code>	Creare siti web semplici
<code>django</code>	Creare siti web completi
<code>pillow</code>	Modificare immagini
<code>pygame</code>	Sviluppare videogiochi 2D
<code>beautifulsoup4</code>	Estrarre dati da pagine web

Buone pratiche

- Usa sempre un **ambiente virtuale** per ogni progetto (vedi il capitolo successivo)

- Tieni aggiornato il file `requirements.txt` quando aggiungi librerie
 - Prima di installare una libreria sconosciuta, verifica che sia affidabile: guarda quante stelle ha su GitHub e quanti download ha su PyPI
-

Ambiente virtuale

Come isolare le dipendenze dei progetti Python con gli ambienti virtuali.

Perché ti serve un ambiente virtuale

Immagina di avere due progetti Python sul tuo computer:

- **Progetto A** funziona con la versione 2.25 della libreria `requests`
- **Progetto B** ha bisogno della versione 2.31 della stessa libreria

Se installi tutto "globalmente" (cioè direttamente sul tuo Python di sistema), i due progetti entrano in conflitto: puoi avere una versione sola, e l'altra non funziona.

Gli **ambienti virtuali** risolvono questo problema: ogni progetto ha la sua "bolla" separata con le proprie librerie, nella versione giusta.

Cos'è un ambiente virtuale?

Un ambiente virtuale è una **cartella isolata** che contiene una copia di Python e di tutte le librerie installate per quel progetto. È separata dal Python del sistema e dagli altri progetti.

Pensa a uno zaino: ogni studente ha il suo, con i propri libri e materiali. Anche se due studenti usano lo stesso manuale, le loro copie sono indipendenti.

Creare un ambiente virtuale

Python include già il modulo `venv` per creare ambienti virtuali. Nella cartella del tuo progetto, esegui:

```
python -m venv venv
```

Questo crea una sottocartella chiamata `venv` con tutto il necessario. Puoi darle qualsiasi nome, ma `venv` o `.venv` sono le convenzioni più usate.

Attivare l'ambiente virtuale

Prima di poterlo usare, devi "attivarlo":

Su macOS e Linux:

```
source venv/bin/activate
```

Su Windows:

```
venv\Scripts\activate
```

Quando l'ambiente è attivo, il nome appare all'inizio del prompt del terminale:

```
(venv) $
```

Da questo momento, qualsiasi `pip install` installerà le librerie **solo in questo ambiente**, non sul sistema globale.

Disattivare l'ambiente virtuale

Quando hai finito di lavorare al progetto:

```
deactivate
```

Il prompt torna alla normalità, senza `(venv)`.

Flusso di lavoro tipico

Ecco la sequenza completa per un nuovo progetto:

```
# 1. Crea la cartella del progetto
mkdir mio-progetto
cd mio-progetto

# 2. Crea l'ambiente virtuale
python -m venv venv

# 3. Attivalo
source venv/bin/activate      # macOS/Linux
# venv\Scripts\activate      # Windows (alternativa)

# 4. Installa le librerie necessarie
pip install requests pandas

# 5. Lavora sul progetto...

# 6. Salva le dipendenze in un file per condividerle
pip freeze > requirements.txt
```

Ripristinare un progetto su un altro computer

Quando qualcuno riceve il tuo progetto (o tu stesso lo riapri su un altro computer):

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt  # Installa tutte le librerie elencate nel
file
```

Non aggiungere `venv` a Git

La cartella `venv` è pesante (può occupare centinaia di megabyte) e si può sempre ricreare con `pip install -r requirements.txt`. Non ha senso caricarla su Git.

Aggiungi queste righe al file `.gitignore` del tuo progetto:

```
venv/  
.venv/  
__pycache__/  
*.pyc
```

Strumenti alternativi (per il futuro)

Strumento	Descrizione
<code>venv</code>	Già incluso in Python — ideale per iniziare
<code>virtualenv</code>	Come <code>venv</code> ma con qualche funzione in più
<code>conda</code>	Molto usato in ambito scientifico (Anaconda)
<code>poetry</code>	Gestisce dipendenze e packaging in modo moderno
<code>uv</code>	Strumento moderno e molto veloce

Per ora, `venv` è più che sufficiente.

Verificare dove sei

```
# Su macOS/Linux: mostra quale Python sta usando
which python

# Su Windows: mostra quale Python sta usando
where python

# Mostra le librerie installate nell'ambiente attivo
pip list
```

Gestione eccezioni avanzata

Tecniche avanzate per gestire le eccezioni in Python.

Ripasso rapido

Hai già visto come usare `try/except` per gestire gli errori. Ora vediamo tecniche più avanzate per situazioni più complesse.

```
try:
    # codice che potrebbe dare errore
except TipoErrore:
    # cosa fare se c'è quell'errore
else:
    # eseguito solo se NON ci sono stati errori
finally:
    # eseguito SEMPRE, con o senza errori
```

Creare i tuoi errori personalizzati

Python ti permette di creare **errori su misura** per il tuo programma. È utile quando vuoi segnalare situazioni specifiche con messaggi chiari.

Pensa a un modulo di iscrizione scolastica: ha senso creare un errore `EtaNonValidaError` invece di usare il generico `ValueError`.

```
# Creo un errore personalizzato per età non valide
class EtaNonValidaError(Exception):
    """Segnala che l'età inserita non è valida."""
    pass

# Creo un errore personalizzato per voti non validi, con informazioni extra
class VotoNonValidoError(Exception):
    """Segnala che il voto inserito non è nell'intervallo 0-10."""
    def __init__(self, voto, messaggio="Il voto deve essere tra 0 e 10"):
        self.voto = voto          # Salvo il voto errato per poterlo
        mostrare                 # Passa il messaggio alla classe
        self.messaggio = messaggio
        super().__init__(self.messaggio)

# Funzioni che usano gli errori personalizzati
def controlla_eta(eta):
    if eta < 0 or eta > 150:
        raise EtaNonValidaError(f"L'età {eta} non è valida.")

def controlla_voto(voto):
    if voto < 0 or voto > 10:
        raise VotoNonValidoError(voto)

# Utilizzo
try:
    controlla_eta(-5)
except EtaNonValidaError as e:
    print(f"Errore: {e}")
# → Errore: L'età -5 non è valida.

try:
    controlla_voto(15)
except VotoNonValidoError as e:
    print(f"Errore: {e.messaggio} (voto ricevuto: {e.voto})")
# → Errore: Il voto deve essere tra 0 e 10 (voto ricevuto: 15)
```

Lanciare un errore manualmente con `raise`

Puoi **provocare** tu stesso un errore quando ti accorgi che qualcosa non va, anche senza che Python lo faccia automaticamente.

```
def dividi(a, b):
    if b == 0:
        raise ValueError("Il divisore non può essere zero.") # Lancio
l'errore
    return a / b

try:
    risultato = dividi(10, 0)
except ValueError as e:
    print(f"Errore: {e}")
# → Errore: Il divisore non può essere zero.
```

È come un cassiere che si rifiuta di accettare una banconota falsa: non aspetta che la banca se ne accorga — interviene subito.

Catturare un errore, fare qualcosa, e rilancerlo

A volte vuoi **registrare** che un errore è avvenuto (ad esempio scriverlo in un file di log), ma poi vuoi comunque che l'errore si propaghi al chiamante.

```
def leggi_file(percorso):
    try:
        with open(percorso) as f:
            return f.read()
    except FileNotFoundError as e:
        print(f"Log: il file '{percorso}' non è stato trovato.")
        raise # Rilancio lo stesso errore — non lo "ingoio"
```

Con `raise` da solo (senza argomenti), Python rilancia esattamente lo stesso errore che hai appena catturato.

Collegare gli errori tra loro (catena di eccezioni)

Puoi indicare che un errore è stato **causato** da un altro errore. Questo aiuta chi legge il messaggio di errore a capire l'intera catena di eventi.

```
def carica_configurazione(filename):
    try:
        with open(filename) as f:
            import json
            return json.load(f)
    except FileNotFoundError as e:
        # Lancio un errore più descrittivo, ma indico che è causato da "e"
        raise RuntimeError(f"Impossibile caricare la configurazione da '{filename}'") from e

try:
    config = carica_configurazione("config.json")
except RuntimeError as e:
    print(f"Errore: {e}")
    print(f"Causato da: {e.__cause__}")
```

La gerarchia degli errori

Gli errori in Python sono organizzati come un albero genealogico. Catturare un errore "genitore" significa catturare anche tutti i suoi "figli".

```
BaseException
├── Exception
│   ├── ArithmeticError
│   │   ├── ZeroDivisionError ← divisione per zero
│   │   └── OverflowError ← numero troppo grande
│   ├── LookupError
│   │   ├── IndexError ← indice lista fuori range
│   │   └── KeyError ← chiave dizionario non trovata
│   ├── ValueError ← valore sbagliato
│   ├── TypeError ← tipo sbagliato
│   └── OSError
│       └── FileNotFoundError ← file non trovato
```

```
try:
    x = [1, 2, 3][10]    # IndexError
except LookupError:
    # Cattura sia IndexError che KeyError, perché entrambi sono "figli" di
    # LookupError
    print("Errore di ricerca (indice o chiave non trovata)")
```

Context manager personalizzato (uso avanzato di `with`)

Hai già usato `with open(...)` per aprire file. Puoi creare i tuoi "context manager" che gestiscono automaticamente l'apertura e la chiusura di risorse — anche in caso di errore.

```
class GestoreRisorsa:
    def __enter__(self):
        # Questo viene eseguito quando si entra nel blocco "with"
        print("Risorsa aperta")
        return self

    def __exit__(self, tipo_errore, valore_errore, traceback):
        # Questo viene eseguito quando si esce dal blocco "with" (anche in
        # caso di errore)
        print("Risorsa chiusa")
        if tipo_errore:
            print(f"Si è verificato un errore: {valore_errore}")
        return False    # False = non "ingoia" l'errore, lascialo propagare

# Utilizzo
with GestoreRisorsa() as r:
    print("Sto usando la risorsa")
    # Se qui avviene un errore, __exit__ viene comunque chiamato
```

Buone pratiche

```
# MALE: cattura tutti gli errori senza distinzione
# Nasconde bug e rende difficile il debug
try:
    operazione_complessa()
except:
    pass # Silenzio totale – non sai cosa è andato storto

# BENE: cattura solo gli errori che ti aspetti
try:
    operazione_complessa()
except (ValueError, TypeError) as e:
    print(f"Errore gestito: {e}")

# BENE: registra l'errore e poi rilancia
try:
    operazione_complessa()
except Exception as e:
    logging.error(f"Errore inatteso: {e}")
    raise # Non nascondere l'errore – rilanciarlo
```

La regola d'oro: **cattura solo gli errori che sai come gestire**. Per tutti gli altri, lascia che si propaghino e vengano visti.

Debugging di base

Come trovare e correggere gli errori nei programmi Python.

Perché ti serve fare debugging

Scrivere codice senza errori al primo colpo è praticamente impossibile — anche per i programmatori più esperti al mondo. Il **debugging** (o "debug") è l'arte di trovare e correggere gli errori nel tuo programma.

Pensa a un detective: quando qualcosa non funziona, devi raccogliere indizi, fare ipotesi e testare le tue teorie finché non trovi il colpevole.

I tre tipi di errori

1. Errori di sintassi

Sono come errori grammaticali: hai scritto qualcosa che Python non capisce. Vengono segnalati **prima ancora che il programma parta**.

```
# Manca i due punti alla fine della riga "if"
if x > 0
    print(x)
# SyntaxError: expected ':'
```

Sono i più semplici da correggere: Python ti dice esattamente dove si trova il problema.

2. Errori durante l'esecuzione

Il codice è scritto correttamente, ma qualcosa va storto **mentre il programma sta girando**.

```
lista = [1, 2, 3]
print(lista[10]) # Stai chiedendo il decimo elemento, ma ne esistono solo 3!
# IndexError: list index out of range
```

È come chiedere alla cassiera di un negozio il prodotto numero 10, ma sullo scaffale ce ne sono solo 3.

3. Errori logici

Questi sono i più insidiosi: il programma **gira senza errori**, ma produce risultati sbagliati. Python non se ne accorge — sei tu che devi trovarli.

```
def calcola_media(numeri):  
    return sum(numeri) / len(numeri) - 1 # Bug! Il "-1" non dovrebbe  
    esserci  
  
print(calcola_media([8, 7, 9])) # Stampa 7.0 invece di 8.0 – sbagliato!
```

Leggere il messaggio di errore (Traceback)

Quando Python trova un errore, stampa un **traceback** — una specie di "mappa" che ti mostra cosa stava succedendo nel momento dell'errore.

```
Traceback (most recent call last):  
  File "script.py", line 10, in <module>  
    risultato = dividi(10, 0)  
  File "script.py", line 4, in dividi  
    return a / b  
ZeroDivisionError: division by zero
```

Come leggerlo: parti dall'ultima riga e vai su.

- **Ultima riga:** il tipo di errore e la descrizione (`ZeroDivisionError: division by zero`)
- **Righe precedenti:** la sequenza di passaggi che ha portato all'errore (prima è stato chiamato `dividi`, che è a riga 10, e dentro `dividi` l'errore è a riga 4)

Tecnica 1: usare `print()` come spia

Il modo più semplice per capire cosa sta succedendo dentro il tuo programma è inserire delle `print()` temporanee per "spiare" i valori delle variabili.

```
def calcola_media(voti):
    print(f"DEBUG: voti ricevuti → {voti}")    # Spia: cosa arriva alla
    funzione?
    totale = sum(voti)
    print(f"DEBUG: totale calcolato → {totale}") # Spia: il totale è
    giusto?
    media = totale / len(voti)
    print(f"DEBUG: media calcolata → {media}")  # Spia: la media è
    giusta?
    return media

risultato = calcola_media([8, 7, 9])
print(f"Risultato finale: {risultato}")
```

Quando hai trovato il bug e corretto il codice, **ricordati di eliminare le righe di debug** prima di consegnare o condividere il programma.

Tecnica 2: il modulo **logging**

Le `print()` funzionano, ma hanno un problema: devi ricordarti di eliminarle tutte. Il modulo `logging` è più elegante: puoi **accenderlo o spegnerlo** con una sola riga.

```
# Questa riga "accende" tutti i messaggi di debug
logging.basicConfig(level=logging.DEBUG)

def calcola_media(voti):
    logging.debug(f"Voti ricevuti: {voti}")
    totale = sum(voti)
    logging.debug(f"Totale: {totale}")
    return totale / len(voti)

calcola_media([8, 7, 9])
```

I livelli di gravità, dal meno al più grave:

Livello	Quando usarlo
DEBUG	Dettagli per il debug
INFO	Informazioni generali ("il programma è partito")
WARNING	Qualcosa di strano, ma il programma continua
ERROR	Qualcosa è andato storto
CRITICAL	Errore gravissimo

Tecnica 3: il debugger interattivo `pdb`

Immagina di poter **mettere in pausa** il tuo programma nel mezzo dell'esecuzione e guardare dentro. È esattamente quello che fa `pdb`.

```
def funzione_problematica(x, y):
    breakpoint() # Il programma si ferma qui e aspetta i tuoi comandi
    risultato = x / y
    return risultato

funzione_problematica(10, 2)
```

Quando il programma si ferma, puoi scrivere comandi:

Comando	Significato
<code>n</code>	Esegui la riga successiva (next)
<code>s</code>	Entra dentro una funzione (step)
<code>p nome_variabile</code>	Stampa il valore di una variabile
<code>c</code>	Continua fino al prossimo punto di pausa (continue)
<code>q</code>	Esci dal debugger (quit)

Il comando `breakpoint()` è disponibile da Python 3.7 in poi. Nelle versioni precedenti si usava `import pdb; pdb.set_trace()`.

Tecnica 4: il debugger grafico (VS Code o PyCharm)

Se usi un editor come **VS Code** o **PyCharm**, hai a disposizione un debugger grafico molto più comodo di `pdb` :

- Clicca sul numero di riga per aggiungere un **punto di pausa** (breakpoint)
- Avanza riga per riga con i pulsanti
- Guarda tutte le variabili in una finestra apposita, senza dover digitare nulla

Trucchi utili

Controlla il tipo di una variabile

A volte il bug è che una variabile contiene un tipo di dato inaspettato (ad esempio una stringa invece di un numero).

```
x = qualche_funzione()
print(type(x), x)    # Stampa sia il tipo che il valore
```

Usa `assert` per aggiungere controlli

`assert` ti permette di scrivere "mi aspetto che questa condizione sia vera — se non lo è, fermati e dimmi perché".

```
def calcola_area(base, altezza):
    assert base > 0, f"La base deve essere positiva, ma ho ricevuto: {base}"
    assert altezza > 0, f"L'altezza deve essere positiva, ma ho ricevuto: {altezza}"
    return base * altezza

calcola_area(5, 3)      # Funziona, area = 15
calcola_area(-1, 3)     # AssertionError: La base deve essere positiva, ma ho ricevuto: -1
```

Isola il problema

Se il bug è in un programma lungo, crea un piccolo programma separato che riproduce solo il problema. Questo ti aiuta a:

- Eliminare tutto il codice non rilevante
- Concentrarti solo sul pezzo problematico
- Testare soluzioni più rapidamente

Gestione degli errori

Come evitare che il programma si blocchi quando qualcosa va storto.

Cosa succede quando si verifica un errore?

Ogni tanto il programma incontra una situazione che non riesce a gestire: l'utente scrive del testo quando ci si aspetta un numero, si cerca di dividere per zero, si prova ad aprire un file che non esiste.

In questi casi Python "lancia" un **errore** (chiamato tecnicamente *eccezione*) e, se non lo gestisci, il programma si blocca e mostra un messaggio di errore.

Esempi di errori comuni:

```
print(10 / 0)           # ZeroDivisionError – non si può dividere per
                        # zero
print("ciao" + 5)       # TypeError – non puoi sommare testo e numero
int("pippo")           # ValueError – "pippo" non è un numero
print(variabile_che_non_c_e) # NameError – variabile non definita

lista = [1, 2, 3]
print(lista[10])        # IndexError – la lista non ha la posizione 10
```

Il blocco `try / except`

Per evitare che il programma si blocchi, "avvolgi" il codice rischioso in un blocco `try`. Se si verifica un errore, Python salta al blocco `except` invece di fermarsi:

```
try:
    risultato = 10 / 0
except ZeroDivisionError:
    print("Non puoi dividere per zero!")
```

Traduzione in parole: "**Prova** a eseguire questo codice. Se si verifica un `ZeroDivisionError`, **invece** fai questo."

Gestire più tipi di errori

Puoi avere più blocchi `except`, uno per ogni tipo di errore che vuoi gestire:

```
try:
    numero = int(input("Inserisci un numero: "))
    risultato = 10 / numero
    print(f"10 / {numero} = {risultato}")
except ValueError:
    print("Devi inserire un numero valido, non del testo.")
except ZeroDivisionError:
    print("Non puoi dividere per zero.")
```

Il blocco `else` — "tutto è andato bene"

Il blocco `else` viene eseguito **solo se non si è verificato nessun errore**:

```
try:
    numero = int(input("Inserisci un numero: "))
    risultato = 10 / numero
except (ValueError, ZeroDivisionError) as e:
    print(f"Errore: {e}")
else:
    print(f"Risultato: {risultato}") # viene eseguito solo se tutto è ok
```

Il blocco **finally** — "sempre, in ogni caso"

Il blocco **finally** viene **sempre eseguito**, che ci sia stato un errore o no. Torna utile per fare "pulizia" (chiudere un file, liberare risorse):

```
try:
    file = open("esempio.txt", "r")
    contenuto = file.read()
except FileNotFoundError:
    print("File non trovato.")
finally:
    print("Operazione completata (con o senza errori).")
```

Leggere il messaggio di errore

Puoi salvare il messaggio di errore con **as** e stamparlo:

```
try:
    x = int("pippo")
except ValueError as e:
    print(f"Dettaglio errore: {e}")
    # Dettaglio errore: invalid literal for int() with base 10: 'pippo'
```

Gli errori più comuni

Errore	Quando succede
<code>ValueError</code>	Valore sbagliato per il tipo (es. <code>int("abc")</code>)
<code>TypeError</code>	Tipo sbagliato (es. <code>"ciao" + 5</code>)
<code>ZeroDivisionError</code>	Divisione per zero
<code>NameError</code>	Variabile non definita
<code>IndexError</code>	Indice fuori dalla lista
<code>KeyError</code>	Chiave non trovata nel dizionario
<code>FileNotFoundError</code>	File non trovato

Esempio pratico: input sicuro

```
while True:
    try:
        a = float(input("Numeratore: "))
        b = float(input("Denominatore: "))
        risultato = a / b
    except ValueError:
        print("Inserisci solo numeri validi.")
        continue
    except ZeroDivisionError:
        print("Non si può dividere per zero.")
        continue
    else:
        print(f"Risultato: {risultato}")
        break
```

Condizioni in Python

Come insegnare al programma a prendere decisioni con if, elif ed else.

Perché le condizioni?

Un programma che fa sempre le stesse cose, senza mai adattarsi alla situazione, è poco utile. Le **condizioni** permettono al programma di prendere decisioni: fare una cosa se qualcosa è vero, un'altra cosa se è falso.

È come un semaforo: se la luce è verde, vai; se è rossa, fermati.

Il blocco **if** — "se"

La parola **if** significa "se" in inglese. Dice a Python: "esegui questo blocco di codice solo se la condizione è vera".

```
if condizione:
    # questo codice viene eseguito solo se la condizione è vera
```

Esempio pratico:

```
eta = 18

if eta >= 18:
    print("Sei maggiorenne.")
```

Se **eta** vale 18 o più, viene stampato il messaggio. Se vale meno di 18, non succede nulla.

:::tip[Ricorda l'indentazione!] Il codice dentro l' **if** deve essere rientrato di 4 spazi. Questa è la regola fondamentale di Python. :::

Il blocco **else** — "altrimenti"

else significa "altrimenti". Si usa per definire cosa fare quando la condizione dell' **if** è falsa:

```
eta = 15

if eta >= 18:
    print("Sei maggiorenne.")
else:
    print("Sei minorenne.")
```

Python controlla la condizione. Se è vera, esegue il blocco `if`. Se è falsa, esegue il blocco `else`. Non possono essere eseguiti entrambi.

Il blocco `elif` — "altrimenti se"

`elif` è l'abbreviazione di "else if" (altrimenti se). Serve quando hai più di due possibilità da gestire:

```
voto = 75

if voto >= 90:
    print("Ottimo!")
elif voto >= 70:
    print("Buono!")
elif voto >= 60:
    print("Sufficiente.")
else:
    print("Insufficiente.")
```

Python controlla le condizioni dall'alto verso il basso e si ferma alla **prima** che è vera. Le altre vengono saltate.

Con `voto = 75` :

1. `voto >= 90` ? No (75 non è >= 90)
2. `voto >= 70` ? Sì! → stampa "Buono!" e si ferma

Condizioni annidate

Puoi mettere un `if` dentro un altro `if`. Si chiama annidamento:

```
eta = 20
ha_patente = True

if eta >= 18:
    if ha_patente:
        print("Puoi guidare.")
    else:
        print("Sei maggiorenne ma non hai la patente.")
else:
    print("Sei minorenne, non puoi guidare.")
```

Ogni livello di annidamento aggiunge altri 4 spazi di indentazione. Tieni d'occhio gli spazi!

Combinare condizioni con **and** e **or**

Invece di annidare, spesso puoi combinare due condizioni in una sola riga:

```
eta = 20
ha_patente = True

# Con 'and': entrambe devono essere vere
if eta >= 18 and ha_patente:
    print("Puoi guidare.")

# Con 'or': basta che una sia vera
eta = 16
con_accompagnatore = True

if eta >= 18 or con_accompagnatore:
    print("Puoi entrare.")
```

Controllare se un valore è in una lista

L'operatore **in** è molto comodo per verificare se un elemento è presente in una sequenza:

```
frutta_disponibile = ["mela", "banana", "uva"]
scelta = "mela"

if scelta in frutta_disponibile:
    print("Ottima scelta, ce l'abbiamo!")
else:
    print("Mi dispiace, quella frutta non c'è.")
```

Esempio completo

```
numero = int(input("Inserisci un numero: "))

if numero > 0:
    print("Il numero è positivo.")
elif numero < 0:
    print("Il numero è negativo.")
else:
    print("Il numero è zero.")
```

Con questo programma:

- Se scrivi `5` → "Il numero è positivo."
- Se scrivi `-3` → "Il numero è negativo."
- Se scrivi `0` → "Il numero è zero."

Cicli for

Come ripetere un'azione per ogni elemento di una lista o sequenza.

Perché usare i cicli?

Immagina di dover salutare 100 studenti uno per uno. Senza i cicli, dovresti scrivere `print("Ciao!")` cento volte. Con un ciclo, lo scrivi una volta sola e Python lo ripete quante volte vuoi.

Il ciclo `for` in particolare è perfetto quando sai già **cosa** vuoi scorrere: una lista di nomi, le lettere di una parola, i numeri da 1 a 10.

Come funziona

```
for elemento in sequenza:  
    # codice da eseguire per ogni elemento
```

Ad ogni giro del ciclo, la variabile `elemento` assume il valore successivo della sequenza. Quando la sequenza finisce, il ciclo si ferma.

Scorrere una lista

L'uso più comune: fare qualcosa per ogni elemento di una lista.

```
frutta = ["mela", "banana", "ciliegia"]  
  
for frutto in frutta:  
    print(frutto)
```

Output:

```
mela  
banana  
ciliegia
```

La variabile `frutto` (che hai scelto tu) prende il valore `"mela"` al primo giro, `"banana"` al secondo, `"ciliegia"` al terzo.

Scorrere una stringa

Una stringa è una sequenza di caratteri, quindi puoi scorrerne le lettere una ad una:

```
for carattere in "Python":  
    print(carattere)
```

Output:

```
P  
y  
t  
h  
o  
n
```

Ripetere un numero preciso di volte con `range()`

Se vuoi eseguire qualcosa esattamente N volte, usa `range()` :

```
for i in range(5):  
    print("Ciao!")
```

Questo stampa "Ciao!" 5 volte. La variabile `i` vale 0, poi 1, poi 2, poi 3, poi 4.

Vedi la sezione su `range()` per tutti i dettagli su come usarla.

Sapere anche la posizione con `enumerate()`

A volte, mentre scorri una lista, vuoi sapere sia il valore sia la sua posizione (indice). Usa `enumerate()` :

```
frutta = ["mela", "banana", "ciliegia"]  
  
for indice, frutto in enumerate(frutta):  
    print(f"{indice}: {frutto}")
```

Output:

```
0: mela
1: banana
2: ciliegia
```

Puoi anche scegliere da quale numero partire:

```
for numero, frutto in enumerate(frutta, start=1):
    print(f"{numero}. {frutto}")
```

Output:

```
1. mela
2. banana
3. ciliegia
```

Scorrere due liste insieme con `zip()`

`zip()` abbina gli elementi di due liste, uno a uno:

```
nomi = ["Alice", "Bob", "Carlo"]
voti = [8, 7, 9]

for nome, voto in zip(nomi, voti):
    print(f"{nome} ha preso {voto}")
```

Output:

```
Alice ha preso 8
Bob ha preso 7
Carlo ha preso 9
```

Interrompere il ciclo: **break** e **continue**

break — ferma il ciclo immediatamente:

```
for i in range(10):
    if i == 5:
        break          # quando i vale 5, esci dal ciclo
    print(i)
# stampa: 0, 1, 2, 3, 4
```

continue — salta questo giro e va al prossimo:

```
for i in range(10):
    if i % 2 == 0:
        continue      # se il numero è pari, saltalo
    print(i)
# stampa solo i numeri dispari: 1, 3, 5, 7, 9
```

Cicli dentro cicli (annidati)

Puoi mettere un ciclo **for** dentro un altro. Per esempio, per costruire una tabellina:

```
for i in range(1, 4):
    for j in range(1, 4):
        print(f"{i} x {j} = {i * j}")
    print("----")
```

Output:

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
---
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
---
...
```

Scorrere un dizionario

I dizionari possono essere scorsi in modi diversi:

```
persona = {"nome": "Alice", "eta": 16, "citta": "Roma"}

# Solo le chiavi
for chiave in persona:
    print(chiave)    # nome, eta, citta

# Chiavi e valori insieme
for chiave, valore in persona.items():
    print(f"{chiave}: {valore}")
```

La funzione range()

Come generare sequenze di numeri per i cicli in Python.

Cos'è `range()` ?

`range()` è una funzione di Python che genera una **sequenza di numeri interi**. Si usa moltissimo nei cicli `for` quando vuoi ripetere qualcosa un numero preciso di volte, o quando vuoi lavorare con numeri in sequenza.

Pensa a `range()` come a un contatore automatico: tu gli dici da dove partire, dove fermarsi e di quanto saltare ad ogni passo.

Le tre forme di `range()`

Solo il numero finale: `range(fine)`

Genera numeri da `0` fino a `fine - 1` (il numero finale non è incluso):

```
for i in range(5):  
    print(i)
```

Output:

```
0  
1  
2  
3  
4
```

:::note[Perché parte da 0?] In programmazione si conta quasi sempre da 0. È una convenzione che incontrerai spesso.

`range(5)` genera 5 numeri, partendo da 0. :::

Inizio e fine: `range(inizio, fine)`

Genera numeri da `inizio` fino a `fine - 1`:

```
for i in range(2, 7):  
    print(i)
```

Output:

```
2  
3  
4  
5  
6
```

Il numero finale (7) non è incluso. È come dire "da 2 incluso a 7 escluso".

Con il passo: `range(inizio, fine, passo)`

Il terzo parametro indica di quanto avanzare ad ogni passo:

```
# Solo i numeri pari da 0 a 10
for i in range(0, 11, 2):
    print(i)
```

Output:

```
0
2
4
6
8
10
```

Il passo può anche essere negativo, per contare al contrario:

```
# Conto alla rovescia da 5 a 1
for i in range(5, 0, -1):
    print(i)

print("Via!")
```

Output:

```
5
4
3
2
1
Via!
```

Convertire range in una lista

Se vuoi vedere tutti i numeri generati da un `range`, convertilo in lista con `list()` :

```
print(list(range(5)))      # [0, 1, 2, 3, 4]
print(list(range(1, 6)))   # [1, 2, 3, 4, 5]
print(list(range(0, 10, 2))) # [0, 2, 4, 6, 8]
```

Esempi pratici

Sommare i numeri da 1 a 100

```
totale = 0
for i in range(1, 101): # da 1 a 100 incluso
    totale += i

print(totale) # 5050
```

La tabellina di un numero

```
n = 7
for i in range(1, 11):
    print(f"{n} x {i} = {n * i}")
```

Output:

```
7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70
```

Accedere agli elementi di una lista tramite posizione

```
frutta = ["mela", "banana", "ciliegia"]

for i in range(len(frutta)):
    print(f"Posizione {i}: {frutta[i]}")
```

Output:

```
Posizione 0: mela
Posizione 1: banana
Posizione 2: ciliegia
```

:::tip Nella maggior parte dei casi, è più semplice e leggibile usare `enumerate()` invece di `range(len(...))`. Vedi il capitolo sui cicli `for`. :::

Cicli while

Come ripetere un'azione finché una condizione rimane vera.

Il ciclo `while` — "finché"

Il ciclo `while` ripete un blocco di codice **finché una condizione rimane vera**. È diverso dal `for`: non sai in anticipo quante volte ripeterà, dipende da cosa succede durante l'esecuzione.

Pensa a un ascensore: le porte rimangono aperte finché non premi il pulsante di un piano. Non sai in anticipo quando lo premeranno.

```
while condizione:
    # codice da ripetere
```

Un primo esempio

```
contatore = 1

while contatore <= 5:
    print(contatore)
    contatore += 1 # aumenta il contatore di 1 ogni giro
```

Output:

```
1
2
3
4
5
```

Come funziona:

1. Python controlla: `contatore <= 5` ? Sì ($1 \leq 5$) → esegue il blocco
2. Stampa `1` , poi `contatore` diventa `2`
3. Controlla ancora: `2 <= 5` ? Sì → stampa `2` , `contatore` diventa `3`
4. ... e così via fino a che `contatore` diventa `6`
5. `6 <= 5` ? No → il ciclo si ferma

Attenzione al ciclo infinito!

Se la condizione non diventa mai falsa, il ciclo non finisce mai. Il programma si blocca per sempre:

```
# NON eseguire questo codice!
x = 0
while x < 10:
    print(x)
    # manca x += 1 - x non cambia mai, la condizione è sempre vera!
```

Se ti capita accidentalmente, puoi fermare il programma con **Ctrl+C** nel terminale.

Interrompere con **break**

break ferma il ciclo immediatamente, anche se la condizione è ancora vera. È utile quando vuoi uscire dal ciclo in base a qualcosa che succede dentro:

```
while True:          # questo ciclo non finirebbe mai da solo...
    risposta = input("Digita 'esci' per uscire: ")
    if risposta == "esci":
        break        # ...ma break lo ferma quando l'utente digita "esci"

print("Programma terminato.")
```

Saltare un giro con **continue**

continue salta il resto del codice in questo giro e passa subito al prossimo controllo della condizione:

```
contatore = 0

while contatore < 10:
    contatore += 1
    if contatore % 2 == 0: # se il numero è pari...
        continue        # ...salta la stampa e vai al giro successivo
    print(contatore)     # stampa solo i numeri dispari
```

Output:

```
1
3
5
7
9
```

Esempi pratici

Chiedere un input fino a quando è valido

Un uso classico del `while` : continuare a chiedere all'utente di inserire qualcosa finché non inserisce un valore accettabile.

```
while True:
    risposta = input("Inserisci 'sì' o 'no': ")
    if risposta == "sì" or risposta == "no":
        break
    print("Non ho capito, riprova.")

print(f"Hai scelto: {risposta}")
```

Conto alla rovescia

```
n = 5

while n > 0:
    print(n)
    n -= 1

print("Via!")
```

Output:

```
5
4
3
2
1
Via!
```

Quando usare `while` invece di `for` ?

- Usa `for` quando sai già **su cosa** vuoi iterare (una lista, un range, ecc.)
 - Usa `while` quando vuoi ripetere qualcosa **finché una condizione cambia**, senza sapere in anticipo quante volte
-

Funzioni in Python

Come creare blocchi di codice riutilizzabili per non ripetere sempre le stesse cose.

Perché usare le funzioni?

Immagina di dover calcolare la media dei voti tre volte in un programma. Senza funzioni, dovresti scrivere la stessa formula tre volte. Se poi vuoi cambiare come si calcola la media, devi ricordarti di cambiarla in tutti e tre i posti.

Con le funzioni, scrivi il calcolo **una volta sola**, gli dai un nome, e lo riusi quante volte vuoi. Se cambia qualcosa, modifichi solo la funzione.

Creare una funzione

Si usa la parola chiave `def` (abbreviazione di "define", definire):

```
def saluta():  
    print("Ciao!")
```

Questa funzione si chiama `saluta` e, quando viene eseguita, stampa "Ciao!". Ma attenzione: definire la funzione non la esegue ancora.

Chiamare una funzione

Per eseguire la funzione, la "chiami" scrivendone il nome seguito dalle parentesi:

```
saluta() # esegue la funzione → stampa "Ciao!"
saluta() # puoi richiamarla quante volte vuoi
saluta()
```

Passare informazioni: i parametri

Le funzioni diventano molto più utili quando possono ricevere informazioni dall'esterno. Queste informazioni si chiamano **parametri**:

```
def saluta(nome):
    print(f"Ciao, {nome}!")

saluta("Alice")    # Ciao, Alice!
saluta("Bob")      # Ciao, Bob!
saluta("Carlo")    # Ciao, Carlo!
```

La stessa funzione si comporta in modo diverso in base a cosa le passi. Puoi avere più parametri:

```
def somma(a, b):
    print(a + b)

somma(3, 5)        # 8
somma(10, 20)      # 30
```

Restituire un risultato: **return**

Finora le funzioni hanno solo stampato qualcosa. Ma spesso vuoi che la funzione **calcoli qualcosa e ti restituisca il risultato**, in modo da poterlo usare dopo.

Si usa la parola chiave `return` :

```
def somma(a, b):
    return a + b

risultato = somma(3, 5)
print(risultato)          # 8
print(somma(10, 20) * 2) # 60 – puoi usarlo direttamente nelle espressioni
```

Quando Python incontra `return`, esce dalla funzione e consegna il valore. Se una funzione non ha `return`, restituisce `None` automaticamente.

Valori predefiniti per i parametri

Puoi assegnare un valore di default a un parametro. Se non viene passato, usa quel valore:

```
def saluta(nome, saluto="Ciao"):
    print(f"{saluto}, {nome}!")

saluta("Alice")          # Ciao, Alice! – usa il default
saluta("Bob", "Buongiorno") # Buongiorno, Bob! – sovrascrive il default
```

I parametri con valore di default devono sempre venire **dopo** quelli senza.

Passare argomenti per nome

Puoi specificare il nome di ogni parametro quando chiami la funzione. L'ordine diventa irrilevante:

```
def descrivi_persona(nome, eta, citta):
    print(f"{nome} ha {eta} anni e vive a {citta}.")

# Ordine normale
descrivi_persona("Alice", 16, "Roma")

# Con i nomi – l'ordine non importa più
descrivi_persona(eta=16, citta="Roma", nome="Alice")
```

Restituire più valori

Una funzione può restituire più di un valore contemporaneamente:

```
def min_max(numeri):  
    return min(numeri), max(numeri)  
  
piccolo, grande = min_max([3, 1, 4, 1, 5, 9])  
print(piccolo)  # 1  
print(grande)   # 9
```

Esempio pratico

```
def calcola_media(voti):  
    if len(voti) == 0:  
        return 0  
    return sum(voti) / len(voti)  
  
def classifica_voto(media):  
    if media >= 9:  
        return "Ottimo"  
    elif media >= 7:  
        return "Buono"  
    elif media >= 6:  
        return "Sufficiente"  
    else:  
        return "Insufficiente"  
  
voti_alice = [8, 7, 9, 6, 10]  
media = calcola_media(voti_alice)  
giudizio = classifica_voto(media)  
  
print(f"Media: {media:.1f} – {giudizio}")  
# Media: 8.0 – Buono
```

Nota come il programma principale è facile da leggere: capisce cosa fa senza dover sapere come funziona ogni funzione internamente.

Funzioni lambda

Funzioni piccole e veloci da scrivere in una sola riga.

Cosa sono le funzioni lambda?

Quando hai bisogno di una funzione semplice, che fa solo una cosa, e non vuoi sprecare righe di codice per definirla con `def`, puoi usare una **funzione lambda**.

È come una funzione normale, ma compressa in una singola riga, senza nome e senza `return` esplicito.

La forma base

```
lambda parametri: espressione
```

Confronta una funzione normale con la sua versione lambda:

```
# Funzione normale
def raddoppia(x):
    return x * 2

# La stessa cosa come lambda
raddoppia_lambda = lambda x: x * 2

print(raddoppia(5))          # 10
print(raddoppia_lambda(5))   # 10
```

Puoi anche avere più parametri:

```
somma = lambda a, b: a + b
print(somma(3, 5))  # 8
```

Quando usare le lambda?

Le lambda sono utili quando devi passare una "piccola funzione" come argomento a un'altra funzione. Il caso più comune è con `sorted()`, `map()` e `filter()`.

Ordinare una lista di oggetti complessi

Se hai una lista di tuple (coppie di valori) e vuoi ordinarla in base al secondo elemento, la lambda è perfetta:

```
studenti = [  
    ("Alice", 16),  
    ("Bob", 15),  
    ("Carlo", 17)  
]  
  
# Ordina per età (il secondo elemento di ogni tupla)  
ordinati = sorted(studenti, key=lambda s: s[1])  
print(ordinati)  
# [('Bob', 15), ('Alice', 16), ('Carlo', 17)]
```

Trasformare ogni elemento di una lista con `map()`

`map()` applica una funzione a ogni elemento di una sequenza:

```
numeri = [1, 2, 3, 4, 5]  
quadrati = list(map(lambda x: x ** 2, numeri))  
print(quadrati) # [1, 4, 9, 16, 25]
```

Equivale a: "per ogni numero nella lista, calcola il suo quadrato".

Filtrare elementi con `filter()`

`filter()` tiene solo gli elementi per cui la funzione restituisce `True`:

```
numeri = [1, 2, 3, 4, 5, 6, 7, 8]
pari = list(filter(lambda x: x % 2 == 0, numeri))
print(pari) # [2, 4, 6, 8]
```

Equivale a: "tieni solo i numeri che, divisi per 2, danno resto zero".

Limiti delle lambda

Le lambda sono semplici per design: possono contenere solo **una singola espressione**. Non puoi usare `if...elif...else` su più righe, cicli, o più istruzioni.

Quando la logica diventa più complessa di una riga, usa sempre una funzione normale con `def` :

```
# Questo va bene con lambda:
doppio = lambda x: x * 2

# Questo è troppo complesso – usa def:
def classifica(voto):
    if voto >= 9:
        return "Ottimo"
    elif voto >= 6:
        return "Sufficiente"
    else:
        return "Insufficiente"
```

Funzioni ricorsive

Funzioni che risolvono un problema chiamando se stesse.

Cos'è la ricorsione?

Una funzione ricorsiva è una funzione che, per risolvere un problema, **chiama se stessa** con una versione più piccola dello stesso problema.

Un'analogia: immagina di dover smontare una bambola matrioska (quelle russe una dentro l'altra). Come la apri? Apri la prima, trovi un'altra matrioska più piccola, la apri, trovi un'altra ancora più piccola... finché non trovi quella

tinyissima che non si apre più. Quella è il **caso base**.

Le due parti fondamentali

Ogni funzione ricorsiva deve avere:

1. Il **caso base** — la condizione che fa fermare la ricorsione
2. Il **caso ricorsivo** — dove la funzione chiama se stessa con un problema più piccolo

Senza il caso base, la funzione si chiamerebbe all'infinito e il programma si bloccherebbe con un errore.

Esempio classico: il fattoriale

Il fattoriale di un numero (scritto con il simbolo **!**) è il prodotto di tutti i numeri interi da 1 fino a quel numero:

```
5! = 5 × 4 × 3 × 2 × 1 = 120
3! = 3 × 2 × 1 = 6
1! = 1
```

Si può anche scrivere così: $5! = 5 \times 4!$. E $4! = 4 \times 3!$. Vedi il pattern? Il problema si riduce sempre.

```
def fattoriale(n):
    # Caso base: se n è 0 o 1, il fattoriale è 1
    if n == 0 or n == 1:
        return 1
    # Caso ricorsivo: n * fattoriale del numero precedente
    return n * fattoriale(n - 1)

print(fattoriale(5))    # 120
print(fattoriale(3))    # 6
```

Ecco cosa succede passo per passo quando chiami **fattoriale(5)** :

```
fattoriale(5)
→ 5 × fattoriale(4)
    → 4 × fattoriale(3)
        → 3 × fattoriale(2)
            → 2 × fattoriale(1)
                → 1 ← caso base, si ferma!
```

Poi si torna indietro calcolando: $2 \times 1 = 2$, $3 \times 2 = 6$, $4 \times 6 = 24$, $5 \times 24 = 120$.

Esempio: la successione di Fibonacci

La successione di Fibonacci è famosa in matematica e in natura: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

Ogni numero è la somma dei due che lo precedono. Anche questo si presta alla ricorsione:

```
def fibonacci(n):
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fibonacci(n - 1) + fibonacci(n - 2)

for i in range(8):
    print(fibonacci(i), end=" ")
# 0 1 1 2 3 5 8 13
```

Un limite importante

Python può gestire al massimo circa 1000 chiamate ricorsive in profondità. Se superi questo limite, ottieni un `RecursionError` . Per questo la ricorsione non è sempre la scelta migliore per problemi con numeri molto grandi.

Quando usare la ricorsione?

La ricorsione è utile quando il problema ha una struttura naturalmente "annidata" — cioè quando si può descrivere un problema grande come una versione più piccola dello stesso problema.

Per problemi semplici (come contare fino a 100, o scorrere una lista), usa i cicli `for` o `while` : sono più veloci e più facili da capire.

Scope delle variabili

Dove le variabili esistono e dove sono accessibili nel codice.

Cos'è lo scope?

Immagina di avere una variabile `x` dentro una funzione. Quella `x` esiste solo dentro quella funzione. Fuori non la puoi usare, come se non esistesse.

Questo concetto si chiama **scope** (o "ambito"): la zona del programma in cui una variabile è accessibile.

Variabili locali — dentro la funzione

Una variabile creata dentro una funzione è **locale**: vive solo per la durata di quella funzione e poi scompare.

```
def mia_funzione():  
    x = 10    # variabile locale: esiste solo dentro questa funzione  
    print(x)  
  
mia_funzione()    # 10  
# print(x)        # NameError: 'x' non esiste qui fuori!
```

Questo è un comportamento voluto, non un difetto. Significa che le funzioni sono **indipendenti**: le variabili dentro una funzione non possono interferire con quelle fuori.

Variabili globali — fuori dalle funzioni

Una variabile definita fuori da qualsiasi funzione è **globale**: è visibile da tutta la parte principale del programma.

```
nome = "Alice"    # variabile globale  
  
def saluta():  
    print(f"Ciao, {nome}!")    # può leggere la variabile globale  
  
saluta()    # Ciao, Alice!
```

Puoi leggere le variabili globali, ma non modificarle (di solito)

Una funzione può **leggere** una variabile globale, ma se prova a modificarla, Python crea una nuova variabile locale con lo stesso nome invece di cambiare quella globale:

```
contatore = 0

def incrementa():
    contatore = 1 # ATTENZIONE: crea una variabile locale, non modifica
    quella globale!
    print(f"Dentro la funzione: {contatore}")

incrementa()
print(f"Fuori dalla funzione: {contatore}") # ancora 0, non è cambiato!
```

Se vuoi davvero modificare una variabile globale dall'interno di una funzione, devi usare la parola chiave `global` :

```
contatore = 0

def incrementa():
    global contatore # ora si riferisce alla variabile globale
    contatore += 1

incrementa()
incrementa()
print(contatore) # 2
```

:::tip[Consiglio] Usare `global` è generalmente una cattiva abitudine. Rende il codice difficile da capire e da correggere. È meglio passare i valori come parametri e usare `return` per restituirli. :::

Variabili con lo stesso nome a livelli diversi

Se hai una variabile locale e una globale con lo stesso nome, dentro la funzione vince quella locale:

```
x = "globale"

def funzione():
    x = "locale"
    print(x)  # locale – la variabile locale ha la precedenza

funzione()
print(x)      # globale – fuori dalla funzione, x è ancora quella globale
```

Non vengono confuse: sono due variabili separate che condividono solo il nome.

Le funzioni predefinite di Python

Funzioni come `print`, `len`, `range`, `input` sono sempre disponibili ovunque nel codice. Python le cerca per ultime, ma le trova sempre.

Per questo non dovresti mai chiamare una tua variabile `print` o `list`: sovrascriveresti la funzione predefinita e poi non potresti più usarla!

```
# Da non fare mai!
# list = [1, 2, 3]   – ora non puoi più usare list() come funzione
# print = "ciao"    – ora print() non funziona più!
```

Interfacce grafiche con Tkinter

Creare finestre e interfacce grafiche semplici con Tkinter in Python.

Perché ti serve Tkinter per le interfacce grafiche

Finora i tuoi programmi comunicano con l'utente tramite il terminale: testo che appare e testo che viene scritto. Con **Tkinter** puoi creare vere finestre grafiche con pulsanti, campi di testo, menu a tendina — come i programmi che usi ogni giorno.

Cos'è Tkinter?

Tkinter è la libreria per creare interfacce grafiche già inclusa in Python. Non devi installare nulla. Permette di creare finestre con tutti i classici elementi: etichette, pulsanti, caselle di testo, caselle di spunta, e molto altro.

La prima finestra

```
# Crea la finestra principale
root = tk.Tk()
root.title("La mia prima finestra")
root.geometry("400x300")  # larghezza × altezza in pixel

# Avvia il "loop degli eventi" – mantiene la finestra aperta e risponde ai
click
root.mainloop()
```

Il metodo `mainloop()` è fondamentale: è il ciclo che mantiene la finestra in vita e reagisce a tutto ciò che l'utente fa (click, digitazione, ecc.).

I componenti principali (Widget)

In Tkinter, ogni elemento visibile si chiama **widget**.

Label — testo fisso

```
root = tk.Tk()

# Crea un'etichetta di testo
etichetta = tk.Label(root, text="Ciao, mondo!", font=("Arial", 16))
etichetta.pack(pady=20) # pack() posiziona il widget nella finestra

root.mainloop()
```

Button — pulsante cliccabile

```
def saluta():
    print("Ciao!") # Questa funzione viene chiamata al click

root = tk.Tk()

pulsante = tk.Button(root, text="Clicca qui!", command=saluta)
pulsante.pack(pady=20)

root.mainloop()
```

`command=saluta` indica quale funzione chiamare quando il pulsante viene premuto.

Entry — campo per inserire testo

```
root = tk.Tk()

# Crea un campo dove l'utente può scrivere
campo = tk.Entry(root, width=30)
campo.pack(pady=10)

def leggi_input():
    testo = campo.get() # Leggi il testo scritto dall'utente
    print(f"Hai scritto: {testo}")

pulsante = tk.Button(root, text="Leggi", command=leggi_input)
pulsante.pack()

root.mainloop()
```

Esempio completo: app di saluto

Metti insieme Label, Entry e Button per fare una piccola app interattiva:

```
def saluta():
    nome = campo_nome.get()
    if nome:
        etichetta_risultato.config(text=f"Ciao, {nome}!")
    else:
        etichetta_risultato.config(text="Inserisci un nome!")

root = tk.Tk()
root.title("App di saluto")
root.geometry("300x200")

# Istruzione per l'utente
tk.Label(root, text="Inserisci il tuo nome:").pack(pady=10)

# Campo di testo
campo_nome = tk.Entry(root, width=25)
campo_nome.pack()

# Pulsante
tk.Button(root, text="Saluta", command=saluta).pack(pady=10)

# Etichetta per mostrare il risultato (inizialmente vuota)
etichetta_risultato = tk.Label(root, text="", font=("Arial", 14))
etichetta_risultato.pack()

root.mainloop()
```

Posizionare i widget in griglia con `grid()`

`pack()` mette i widget uno sotto l'altro. Con `grid()` puoi posizzionarli in una **tabella** di righe e colonne — utile per form o calcolatrici:

```

root = tk.Tk()
root.title("Calcolatrice semplice")

# Riga 0: etichetta e campo per il primo numero
tk.Label(root, text="Primo numero:").grid(row=0, column=0, padx=5, pady=5)
campo_a = tk.Entry(root)
campo_a.grid(row=0, column=1)

# Riga 1: etichetta e campo per il secondo numero
tk.Label(root, text="Secondo numero:").grid(row=1, column=0, padx=5,
pady=5)
campo_b = tk.Entry(root)
campo_b.grid(row=1, column=1)

def calcola():
    try:
        a = float(campo_a.get())
        b = float(campo_b.get())
        etichetta_ris.config(text=f"Somma: {a + b}")
    except ValueError:
        etichetta_ris.config(text="Inserisci numeri validi!")

# Riga 2: pulsante che occupa entrambe le colonne
tk.Button(root, text="Somma", command=calcola).grid(row=2, column=0,
columnspan=2, pady=10)

# Riga 3: etichetta risultato
etichetta_ris = tk.Label(root, text="")
etichetta_ris.grid(row=3, column=0, columnspan=2)

root.mainloop()

```

Altri widget utili

```
root = tk.Tk()

# Text: area di testo su più righe (come un piccolo editor)
area = tk.Text(root, height=5, width=30)
area.pack()

# Checkbutton: casella di spunta
var_check = tk.BooleanVar()
check = tk.Checkbutton(root, text="Accetta i termini", variable=var_check)
check.pack()

# Radiobutton: scelta esclusiva tra opzioni
var_radio = tk.StringVar(value="mela")
for frutto in ["mela", "banana", "uva"]:
    tk.Radiobutton(root, text=frutto, variable=var_radio,
value=frutto).pack()

# Scale: cursore scorrevole
cursore = tk.Scale(root, from_=0, to=100, orient=tk.HORIZONTAL)
cursore.pack()

root.mainloop()
```

Finestre di dialogo

Tkinter ha anche finestre di dialogo già pronte — quelle che compaiono davanti alla finestra principale con un messaggio o una domanda:

```
from tkinter import messagebox

root = tk.Tk()

def mostra_info():
    # Finestra di informazione con un solo pulsante OK
    messagebox.showinfo("Info", "Operazione completata!")

def chiedi_conferma():
    # Finestra con pulsanti Sì e No
    risposta = messagebox.askyesno("Conferma", "Sei sicuro di voler uscire?")
    if risposta:
        root.quit() # Chiudi il programma

tk.Button(root, text="Mostra info", command=mostra_info).pack(pady=5)
tk.Button(root, text="Esci", command=chiedi_conferma).pack(pady=5)

root.mainloop()
```

Grafica con Turtle

Disegnare con la libreria Turtle in Python.

Qual è la funzione di Turtle nell'apprendimento

Turtle è il modo più semplice e divertente per vedere i risultati del tuo codice in modo visivo. Invece di stampare numeri sul terminale, disegni figure sullo schermo — e puoi vedere ogni riga del tuo programma "prendere vita".

È anche un ottimo strumento per capire come funzionano i cicli e la geometria.

Cos'è Turtle?

Turtle è una libreria già inclusa in Python — non devi installare nulla. Ti permette di controllare una "tartaruga" virtuale su uno schermo. La tartaruga ha una penna sotto il guscio: quando si muove, lascia una traccia. Tu dai i comandi e la tartaruga disegna.

L'idea viene dal linguaggio Logo degli anni '60, usato proprio per insegnare la programmazione ai bambini.

Il primo programma

```
t = turtle.Turtle()    # Crea la tartaruga

t.forward(100)          # Avanza di 100 pixel
t.right(90)             # Gira a destra di 90°
t.forward(100)          # Avanza ancora

turtle.done()           # Mantieni la finestra aperta
```

Comandi di movimento

```
t = turtle.Turtle()

t.forward(100)          # Avanza di 100 pixel (abbreviazione: t.fd(100))
t.backward(50)          # Arretra di 50 pixel (abbreviazione: t.bk(50))
t.right(90)             # Gira a destra di 90° (abbreviazione: t.rt(90))
t.left(90)              # Gira a sinistra di 90° (abbreviazione: t.lt(90))
t.goto(100, 50)         # Vai alle coordinate (x, y) – l'origine è al centro
t.home()                # Torna all'origine (0, 0) e guarda a destra
```

Alzare e abbassare la penna

```
t.penup()    # Alza la penna: la tartaruga si muove senza disegnare
t.pendown()  # Abbassa la penna: torna a disegnare
```

Utile per "saltare" in un altro punto senza lasciare tracce.

Personalizzare il disegno

```
t.pensize(3)      # Spessore della linea (in pixel)
t.pencolor("red")  # Colore della linea
t.fillcolor("yellow") # Colore di riempimento (usato con begin_fill /
end_fill)
t.speed(5)         # Velocità: 1=lenta, 10=veloce, 0=massima
(instantanea)
```

Disegnare figure geometriche

Quadrato

```
t = turtle.Turtle()

# Un quadrato ha 4 lati uguali e 4 angoli di 90°
for _ in range(4):
    t.forward(100)  # Disegna un lato
    t.right(90)     # Gira di 90°

turtle.done()
```

Triangolo equilatero

```
t = turtle.Turtle()

# Un triangolo equilatero ha 3 lati e angoli esterni di 120°
for _ in range(3):
    t.forward(100)
    t.left(120)

turtle.done()
```

Cerchio

```
t = turtle.Turtle()

t.circle(50)    # Disegna un cerchio con raggio 50 pixel

turtle.done()
```

Stella a 5 punte

```
t = turtle.Turtle()
t.speed(0)

# Per una stella a 5 punte l'angolo esterno è 144°
for _ in range(5):
    t.forward(100)
    t.right(144)

turtle.done()
```

Riempire una figura con il colore

```
t = turtle.Turtle()

t.fillcolor("blue")    # Scegli il colore di riempimento
t.begin_fill()         # Da qui inizia il riempimento

for _ in range(4):     # Disegna il quadrato
    t.forward(100)
    t.right(90)

t.end_fill()           # Riempi la figura chiusa
turtle.done()
```

Personalizzare la finestra

```
schermo = turtle.Screen()
schermo.title("Il mio disegno")    # Titolo della finestra
schermo.bgcolor("black")           # Colore dello sfondo
schermo.setup(width=800, height=600) # Dimensioni in pixel

t = turtle.Turtle()
t.pencolor("white")    # Disegna in bianco su sfondo nero
t.forward(200)

turtle.done()
```

Esempio: spirale arcobaleno

Combina un ciclo, colori e geometria per creare qualcosa di bello:

```
t = turtle.Turtle()
t.speed(0) # Velocità massima

colori = ["red", "orange", "yellow", "green", "blue", "purple"]

for i in range(100):
    t.pencolor(colori[i % len(colori)]) # Cambia colore ciclicamente
    t.forward(i * 2) # Ogni volta avanza un po' di più
    t.right(91) # 91° invece di 90° crea la
    spirale

turtle.done()
```

Prova a modificare l'angolo (91°) o il numero di passi (100) per ottenere effetti diversi.

Gestione dei file

Come leggere e scrivere file con Python — per salvare e caricare dati.

Perché lavorare con i file?

Quando un programma si chiude, tutto quello che era in memoria scompare. Se vuoi che i dati sopravvivano tra una esecuzione e l'altra (come i punteggi di un gioco, o una lista di nomi), devi **salvarli su file**.

Aprire un file

Per lavorare con un file, lo "apri" con la funzione `open()`. Devi specificare il nome del file e la **modalità** (cosa vuoi fare):

Modalità	Cosa fa
"r"	Legge il file (deve esistere)
"w"	Scriva nel file (lo crea se non esiste, lo svuota se esiste)
"a"	Aggiunge alla fine del file senza cancellare il contenuto

Il modo corretto: il blocco `with`

Usa sempre il costrutto `with` per aprire i file. Garantisce che il file venga chiuso correttamente anche se c'è un errore:

```
with open("esempio.txt", "r") as file:
    contenuto = file.read()
    print(contenuto)
# Il file viene chiuso automaticamente quando esci dal blocco with
```

Leggere un file

Leggere tutto il contenuto in una volta:

```
with open("esempio.txt", "r") as file:
    contenuto = file.read()
    print(contenuto)
```

Leggere riga per riga (utile per file grandi):

```
with open("esempio.txt", "r") as file:
    for riga in file:
        print(riga.strip()) # strip() rimuove il carattere di a capo alla
fine
```

Ottenere tutte le righe come lista:

```
with open("esempio.txt", "r") as file:
    righe = file.readlines() # lista di stringhe, una per riga

print(righe)
```

Scrivere su un file

Creare un nuovo file (o sovrascriverlo):

```
with open("output.txt", "w") as file:
    file.write("Prima riga\n")
    file.write("Seconda riga\n")
    file.write("Terza riga\n")
```

:::caution La modalità `"w"` cancella tutto il contenuto precedente del file. Se vuoi aggiungere senza cancellare, usa `"a"`.

Aggiungere righe a un file esistente:

```
with open("output.txt", "a") as file:
    file.write("Questa riga viene aggiunta alla fine\n")
```

Gestire il caso in cui il file non esiste

Se provi ad aprire un file che non esiste in modalità `"r"`, Python dà un errore. Usa `try/except` per gestirlo:

```
try:
    with open("dati.txt", "r") as file:
        contenuto = file.read()
        print(contenuto)
except FileNotFoundError:
    print("Il file non esiste ancora.")
```

Esempio pratico: salvare e caricare una lista

```
def salva_studenti(studenti, nome_file):
    """Salva la lista degli studenti su file."""
    with open(nome_file, "w") as file:
        for nome, voto in studenti:
            file.write(f"{nome},{voto}\n")

def carica_studenti(nome_file):
    """Carica la lista degli studenti dal file."""
    studenti = []
    try:
        with open(nome_file, "r") as file:
            for riga in file:
                parti = riga.strip().split(",")
                if len(parti) == 2:
                    nome = parti[0]
                    voto = float(parti[1])
                    studenti.append((nome, voto))
    except FileNotFoundError:
        print("File non trovato, parto con lista vuota.")
    return studenti

# Salva i dati su file
studenti = [("Alice", 8.5), ("Bob", 7.0), ("Carlo", 9.2)]
salva_studenti(studenti, "studenti.csv")

# Ricarica i dati dal file
caricati = carica_studenti("studenti.csv")
for nome, voto in caricati:
    print(f"{nome}: {voto}")
```

Il file `studenti.csv` conterrà:

```
Alice,8.5
Bob,7.0
Carlo,9.2
```

Formattazione dell'output

Come presentare i dati in modo leggibile e ordinato.

Perché formattare l'output?

Stampare dati grezzi funziona, ma non è sempre leggibile. Formattare l'output significa presentare le informazioni in modo chiaro: numeri con il numero giusto di cifre decimali, testo allineato in colonne ordinate.

Le f-string: il metodo consigliato

Le **f-string** (disponibili da Python 3.6) sono il modo più comodo e leggibile per costruire stringhe con variabili. Si scrivono mettendo **f** prima delle virgolette:

```
nome = "Alice"
eta = 16
print(f"Mi chiamo {nome} e ho {eta} anni.")
# Mi chiamo Alice e ho 16 anni.
```

Dentro le parentesi graffe puoi scrivere qualsiasi espressione Python:

```
a, b = 5, 3
print(f"{a} + {b} = {a + b}")
print(f"Il quadrato di {a} è {a**2}")
```

Controllare i decimali

Per i numeri decimali puoi specificare quante cifre mostrare dopo la virgola:

```
pi = 3.14159265
print(f"{pi:.2f}")    # 3.14    - 2 cifre decimali
print(f"{pi:.4f}")    # 3.1416  - 4 cifre decimali
print(f"{pi:.0f}")    # 3        - nessuna cifra decimale
```

La sintassi è `{valore:.Nf}` dove **N** è il numero di cifre decimali.

Percentuali

```
percentuale = 0.75
print(f"{percentuale:.0%}")    # 75%
print(f"{percentuale:.2%}")    # 75.00%
```

Python moltiplica per 100 e aggiunge il simbolo `%` automaticamente.

Separatore delle migliaia

Per i numeri grandi, puoi aggiungere la virgola come separatore:

```
numero = 1234567
print(f"{numero:,}")    # 1,234,567
```

Allineare il testo in colonne

Quando vuoi creare tabelle con colonne ordinate, puoi specificare la larghezza e l'allineamento:

```
print(f"{'Sinistra':<15}|")    # allineato a sinistra – 15 caratteri
print(f"{'Centro':^15}|")      # centrato
print(f"{'Destra':>15}|")      # allineato a destra
```

Output:

```
Sinistra      |
      Centro  |
           Destra|
```

Opzioni di `print()`

La funzione `print()` ha due opzioni utili:

sep — il testo che viene messo tra gli argomenti (default: uno spazio):

```
print("mela", "banana", "uva")           # mela banana uva
print("mela", "banana", "uva", sep=", ")  # mela, banana, uva
print("mela", "banana", "uva", sep=" | ") # mela | banana | uva
```

end — il testo aggiunto alla fine (default: va a capo `\n`):

```
print("Ciao", end=" ")
print("mondo!")           # Ciao mondo! — sulla stessa riga
```

Utile per stampare più elementi sulla stessa riga in un ciclo:

```
for i in range(5):
    print(i, end=" ")
print() # va a capo alla fine
# Output: 0 1 2 3 4
```

Esempio: una tabella formattata

```
studenti = [  
    ("Alice", 16, 8.5),  
    ("Bob", 15, 7.0),  
    ("Carlo", 17, 9.2),  
]  
  
# Intestazione della tabella  
print(f"{'Nome':<10} {'Età':>5} {'Media':>8}")  
print("-" * 25)  
  
# Righe della tabella  
for nome, eta, media in studenti:  
    print(f"{'nome':<10} {'eta':>5} {'media':>8.1f}")
```

Output:

Nome	Età	Media
Alice	16	8.5
Bob	15	7.0
Carlo	17	9.2

Input utente

Come chiedere informazioni all'utente durante l'esecuzione del programma.

Perché l'input è importante?

Un programma che fa sempre le stesse cose con gli stessi dati è limitato. Con l'**input utente**, il programma può fare domande e comportarsi in modo diverso in base alle risposte. Questo è il primo passo per creare programmi davvero interattivi.

La funzione `input()`

`input()` mostra un messaggio e poi si mette in attesa: il programma si ferma finché l'utente non scrive qualcosa e preme Invio.

```
nome = input("Come ti chiami? ")
print(f"Ciao, {nome}!")
```

Esempio di esecuzione:

```
Come ti chiami? Alice
Ciao, Alice!
```

Il testo dentro le parentesi di `input()` è il messaggio che viene mostrato all'utente.

Attenzione: `input()` restituisce sempre testo

Anche se l'utente digita un numero, `input()` lo tratta sempre come **testo** (stringa). Se vuoi fare calcoli, devi convertire:

```
# Questo NON funziona per fare calcoli:
eta_testo = input("Quanti anni hai? ")
# eta_testo è una stringa, es. "16" – non il numero 16!

# Converti con int() per avere un numero intero:
eta = int(input("Quanti anni hai? "))
print(f"Tra 10 anni avrai {eta + 10} anni.")

# Converti con float() per numeri decimali:
altezza = float(input("Altezza in metri: "))
print(f"Altezza: {altezza} m")
```

Cosa succede se l'utente sbaglia?

Se l'utente digita "pippo" invece di un numero e tu usi `int()`, il programma crasha con un errore.

Per gestire questo caso elegantemente, usa `try/except` :

```
try:
    numero = int(input("Inserisci un numero: "))
    print(f"Hai inserito: {numero}")
except ValueError:
    print("Quello non era un numero! Riprova.")
```

Continuare a chiedere finché la risposta è valida

Un pattern molto comune: ripetere la domanda finché l'utente non inserisce un valore accettabile.

```
while True:
    try:
        eta = int(input("Inserisci la tua età: "))
        if eta < 0 or eta > 120:
            print("Età non valida. Inserisci un numero tra 0 e 120.")
            continue # riparte dall'inizio del ciclo
        break # esce dal ciclo se tutto è ok
    except ValueError:
        print("Scrivi un numero, non del testo.")

print(f"La tua età è: {eta}")
```

Esempio pratico: calcolatrice

```
print("Calcolatrice semplice")
print("-" * 20)

try:
    a = float(input("Primo numero: "))
    operazione = input("Operazione (+, -, *, /): ")
    b = float(input("Secondo numero: "))

    if operazione == "+":
        risultato = a + b
    elif operazione == "-":
        risultato = a - b
    elif operazione == "*":
        risultato = a * b
    elif operazione == "/":
        if b == 0:
            print("Errore: non si divide per zero!")
        else:
            risultato = a / b
    else:
        print("Operazione non riconosciuta.")

    print(f"Risultato: {a} {operazione} {b} = {risultato}")

except ValueError:
    print("Inserisci solo numeri validi.")
```

Date e orari

Lavorare con date e orari usando il modulo datetime in Python.

Perché è utile usare date e orari in modo corretto

Qualunque programma reale prima o poi ha a che fare con le date: "quando è stata inviata questa email?", "quanti giorni mancano al compleanno?", "quanti giorni fa ho creato questo file?". Python ha un modulo dedicato per tutto questo.

Il modulo `datetime`

Il modulo `datetime` offre quattro strumenti principali:

Classe	Cosa rappresenta
<code>date</code>	Solo la data (anno, mese, giorno)
<code>time</code>	Solo l'orario (ore, minuti, secondi)
<code>datetime</code>	Data e orario insieme
<code>timedelta</code>	Una durata — la differenza tra due momenti

```
from datetime import date, datetime, timedelta # Importa solo quello che serve
```

La data e l'ora di adesso

```
from datetime import date, datetime

oggi = date.today()      # Solo la data odierna
print(oggi)              # 2024-01-15 (formato: anno-mese-giorno)

adesso = datetime.now()  # Data e orario attuali
print(adesso)            # 2024-01-15 14:30:25.123456
```

Creare una data specifica

```
from datetime import date, datetime

compleanno = date(2008, 3, 25)          # anno=2008, mese=3, giorno=25
print(compleanno)    # 2008-03-25

capodanno = datetime(2025, 1, 1, 0, 0, 0) # anno, mese, giorno, ora,
print(capodanno)     # 2025-01-01 00:00:00
```

Accedere alle singole parti di una data

```
from datetime import datetime

adesso = datetime.now()

print(adesso.year)      # Anno (es: 2024)
print(adesso.month)     # Mese (1-12)
print(adesso.day)       # Giorno (1-31)
print(adesso.hour)      # Ora (0-23)
print(adesso.minute)    # Minuti (0-59)
print(adesso.second)    # Secondi (0-59)
```

Formattare una data come testo: **strftime()**

Per default le date si stampano nel formato `2024-01-15`. Con `strftime()` puoi scegliere tu come vuoi che appaia:

```
from datetime import datetime

adesso = datetime.now()

print(adesso.strftime("%d/%m/%Y"))      # 15/01/2024 (formato italiano)
print(adesso.strftime("%d %B %Y"))      # 15 January 2024 (con nome del
mese)
print(adesso.strftime("%H:%M:%S"))      # 14:30:25 (solo l'orario)
print(adesso.strftime("%d/%m/%Y %H:%M")) # 15/01/2024 14:30
```

I codici di formato principali:

Codice	Significato	Esempio
%Y	Anno (4 cifre)	2024
%m	Mese (2 cifre)	01
%d	Giorno (2 cifre)	15
%H	Ora in formato 24h	14
%M	Minuti	30
%S	Secondi	25
%A	Nome del giorno (inglese)	Monday
%B	Nome del mese (inglese)	January

Convertire testo in data: `strptime()`

L'operazione inversa: prendi una stringa di testo e trasformala in un oggetto data. Devi dirle in che formato è scritta la stringa:

```
from datetime import datetime

testo = "25/03/2008"
data = datetime.strptime(testo, "%d/%m/%Y") # Dici a Python come è
formattata la stringa
print(data)          # 2008-03-25 00:00:00
print(data.year)     # 2008
```

Calcolare differenze tra date: **timedelta**

timedelta rappresenta una **durata** — un certo numero di giorni, ore, minuti, ecc. Puoi aggiungerla o sottrarla a una data:

```
from datetime import date, timedelta

oggi = date.today()

# Tra quanti giorni sarà la prossima settimana?
tra_una_settimana = oggi + timedelta(days=7)
print(tra_una_settimana)

# Che giorno era 30 giorni fa?
trenta_giorni_fa = oggi - timedelta(days=30)
print(trenta_giorni_fa)

# Quanti giorni sono passati dal tuo compleanno?
nato = date(2008, 3, 25)
differenza = date.today() - nato # Sottrai due date → ottieni un
timedelta
print(f"Giorni vissuti: {differenza.days}")
```

Confrontare date

Le date si confrontano come i numeri — con `<`, `>`, `==` :

```
from datetime import date

d1 = date(2024, 1, 1)
d2 = date(2024, 12, 31)

print(d1 < d2)    # True ← d1 viene prima di d2
print(d1 == d2)   # False ← sono date diverse

if date.today() > d1:
    print("Siamo dopo il 1 gennaio 2024")
```

JSON

Lavorare con il formato JSON in Python.

Qual è la funzione di JSON nelle applicazioni

Ogni volta che un'app sul tuo telefono mostra dati dal web — il meteo, i risultati di una ricerca, i post di un social network — quei dati spesso arrivano in formato **JSON**.

JSON è il "linguaggio comune" che i programmi usano per scambiarsi informazioni. Imparare a lavorare con JSON ti permette di fare cose concrete: scaricare dati da internet, salvarli, leggerli.

Cos'è il JSON?

JSON (si legge "djèizon") è un formato di testo per rappresentare dati strutturati. Assomiglia molto ai dizionari e alle liste Python:

```
{
  "nome": "Alice",
  "eta": 16,
  "voti": [8, 7, 9, 6],
  "indirizzo": {
    "citta": "Roma",
    "cap": "00100"
  },
  "attivo": true
}
```

È leggibile dagli esseri umani (se ben indentato) e facilmente interpretabile dai computer.

Importare il modulo

Da Python a JSON: `json.dumps()`

`dumps()` converte un dizionario (o una lista) Python in una **stringa** JSON. Utile per inviare dati o stamparli.

```

dati = {
    "nome": "Alice",
    "eta": 16,
    "voti": [8, 7, 9],
    "attivo": True,      # Python: True → JSON: true
    "media": None        # Python: None → JSON: null
}

# Converti in stringa JSON (su una riga)
json_stringa = json.dumps(dati)
print(json_stringa)
# → {"nome": "Alice", "eta": 16, "voti": [8, 7, 9], "attivo": true,
"media": null}

# Converti in stringa JSON leggibile (con indentazione)
json_bello = json.dumps(dati, indent=4)
print(json_bello)

```

Output indentato:

```

{
    "nome": "Alice",
    "eta": 16,
    "voti": [8, 7, 9],
    "attivo": true,
    "media": null
}

```

Conversione dei tipi: Python ↔ JSON

Python	JSON
<code>dict</code>	<code>{}</code> (oggetto)
<code>list</code>	<code>[]</code> (array)
<code>str</code>	<code>"..."</code> (stringa)
<code>int</code> , <code>float</code>	numero
<code>True</code> / <code>False</code>	<code>true</code> / <code>false</code>
<code>None</code>	<code>null</code>

Da JSON a Python: `json.loads()`

`loads()` fa il contrario: prende una stringa JSON e la converte in un dizionario (o lista) Python.

```
stringa_json = '{"nome": "Alice", "eta": 16, "voti": [8, 7, 9]}'

dati = json.loads(stringa_json)  # Converti la stringa in dizionario
Python

print(dati["nome"])             # Alice
print(dati["voti"])             # [8, 7, 9]
print(type(dati))               # <class 'dict'>
```

Scrivere JSON su file: `json.dump()`

`dump()` (senza la "s") scrive direttamente su un file:

```

studenti = [
    {"nome": "Alice", "eta": 16, "voto": 8},
    {"nome": "Bob", "eta": 15, "voto": 7},
]

with open("studenti.json", "w") as file:
    json.dump(studenti, file, indent=4) # Scrive il JSON nel file, ben
    indentato

```

Il file `studenti.json` conterrà:

```

[
  {
    "nome": "Alice",
    "eta": 16,
    "voto": 8
  },
  ...
]

```

Leggere JSON da file: `json.load()`

`load()` legge un file JSON e lo converte in Python:

```

with open("studenti.json", "r") as file:
    studenti = json.load(file) # Legge il file e converte in lista Python

for studente in studenti:
    print(f"{studente['nome']}: {studente['voto']}")
# → Alice: 8
# → Bob: 7

```

Gestire errori di formato

Se la stringa JSON non è valida (ad esempio ha una virgola di troppo), Python lancia un errore. Puoi gestirlo con `try/except` :

```
try:
    dati = json.loads("questo non è JSON valido!!!")
except json.JSONDecodeError as e:
    print(f"Errore: il testo non è JSON valido – {e}")
```

Esempio pratico: file di configurazione

Un uso comune di JSON è salvare le impostazioni di un programma in un file che l'utente può modificare:

```
# Salva le impostazioni
config = {
    "tema": "scuro",
    "lingua": "italiano",
    "notifiche": True,
    "volume": 75
}

with open("config.json", "w") as f:
    json.dump(config, f, indent=2)

# La prossima volta che il programma parte, ricarica le impostazioni
with open("config.json", "r") as f:
    config_caricata = json.load(f)

print(f"Tema: {config_caricata['tema']}")      # Tema: scuro
print(f"Lingua: {config_caricata['lingua']}")  # Lingua: italiano
```

Matematica

Operazioni matematiche avanzate con il modulo math in Python.

Qual è la funzione del modulo math

Gli operatori base (`+` , `-` , `*` , `/` , `**`) coprono la maggior parte dei calcoli quotidiani. Ma appena ti servono radici quadrate, logaritmi, seno e coseno, o il valore di pi greco, hai bisogno del modulo `math` .

Per usarlo, basta questa riga all'inizio del tuo programma:

Costanti matematiche

Il modulo `math` include alcune costanti fondamentali già pronte. Eccole tutte:

```
print(math.pi)      # 3.141592653589793 – il pi greco (π)
print(math.e)        # 2.718281828459045 – il numero di Nepero (base dei
logaritmi naturali)
print(math.inf)      # inf – infinito (utile per confronti: nessun numero è
più grande)
print(math.nan)      # nan – "Not a Number", cioè "non è un numero" (risultato
di operazioni impossibili, come 0/0)
```

Arrotondamento

Queste funzioni arrotondano un numero decimale in modi diversi. Guarda come si comportano:

```
print(math.floor(3.7))    # 3 – arrotonda VERSO IL BASSO (piano inferiore)
print(math.ceil(3.2))    # 4 – arrotonda VERSO L'ALTO (piano superiore)
print(round(3.5))        # 4 – arrotondamento standard (già disponibile
                           senza math)
print(math.trunc(3.9))    # 3 – tronca la parte decimale (butta via i
                           decimali)
```

Analogia: pensa ai piani di un edificio. `floor` ti porta al piano di sotto, `ceil` a quello di sopra.

Valore assoluto

```
print(abs(-5))           # 5 – funzione già disponibile senza math
print(math.fabs(-5.0))    # 5.0 – versione di math (restituisce sempre un
                           decimale)
```

Il valore assoluto è la "distanza da zero" — elimina il segno negativo.

Potenze e radici

```
print(math.sqrt(16))     # 4.0 – radice quadrata ( $\sqrt{16} = 4$ )
print(math.sqrt(2))      # 1.4142135... – radice quadrata di 2

print(math.pow(2, 10))   # 1024.0 – potenza ( $2^{10}$ ), equivalente a 2**10 ma
                           restituisce float
print(math.isqrt(17))    # 4 – radice quadrata intera (tronca, senza
                           decimali)
```

Logaritmi e funzioni esponenziali

```
# Esponenziale: e elevato alla x
print(math.exp(1))    # 2.718... - e^1
print(math.exp(2))    # 7.389... - e^2

# Logaritmo naturale (base e)
print(math.log(math.e))    # 1.0 - logaritmo naturale di e

# Logaritmo in base personalizzata
print(math.log(100, 10))    # 2.0 - log base 10 di 100 (quanto è 10^x = 100? Risposta: 2)
print(math.log10(1000))    # 3.0 - scorciatoia per log base 10
print(math.log2(8))        # 3.0 - scorciatoia per log base 2 (2^3 = 8)
```

Trigonometria

Le funzioni trigonometriche lavorano in **radianti**, non in gradi. Python include funzioni di conversione:

```
# Converti gradi ↔ radianti
print(math.radians(180))    # 3.14159... (180 gradi = π radianti)
print(math.degrees(math.pi)) # 180.0

# Seno, coseno, tangente
print(math.sin(math.pi / 2)) # 1.0 - seno di 90°
print(math.cos(0))          # 1.0 - coseno di 0°
print(math.tan(math.pi / 4)) # 1.0 - tangente di 45°
```

Altre funzioni utili

```
print(math.gcd(12, 8))      # 4   – Massimo Comune Divisore (MCD)
print(math.lcm(4, 6))      # 12  – Minimo Comune Multiplo (MCM) [Python
3.9+]
print(math.factorial(5))   # 120 – fattoriale: 5! = 5×4×3×2×1
```

Il modulo `random` : numeri casuali

Spesso usato insieme a `math`, il modulo `random` genera numeri casuali — utile per giochi, simulazioni, e molto altro:

```
print(random.random())      # Numero decimale casuale tra 0.0 e 1.0
print(random.randint(1, 10)) # Numero intero casuale tra 1 e 10
(inclusi)
print(random.uniform(1.0, 5.0)) # Numero decimale casuale tra 1.0 e 5.0

lista = [1, 2, 3, 4, 5]
print(random.choice(lista))  # Elemento scelto a caso dalla lista
random.shuffle(lista)        # Mescola la lista in ordine casuale
print(lista)                 # Esempio: [3, 1, 5, 2, 4]
```

Esempio pratico: calcoli geometrici

```
def area_cerchio(raggio):  
    # Formula:  $\pi \times r^2$   
    return math.pi * raggio ** 2  
  
def ipotenusa(cateto_a, cateto_b):  
    # Teorema di Pitagora:  $c = \sqrt{a^2 + b^2}$   
    return math.sqrt(cateto_a ** 2 + cateto_b ** 2)  
  
print(f"Area cerchio (r=5): {area_cerchio(5):.2f}")      # 78.54  
print(f"Ipotenusa (3, 4): {ipotenusa(3, 4):.2f}")        # 5.00
```

Moduli

Come organizzare il codice in moduli e importarli in Python.

Perché è utile usare i moduli

Immagina di dover scrivere un programma grande. Mettere tutto il codice in un unico file lungo migliaia di righe sarebbe un disastro — impossibile da leggere o da modificare.

I **moduli** sono il modo in cui Python ti permette di suddividere il codice in file separati, organizzati per argomento. Ogni file `.py` è già un modulo. E Python viene fornito con centinaia di moduli già pronti da usare — la cosiddetta **libreria standard**.

Importare un modulo con `import`

Per usare un modulo, devi "importarlo" all'inizio del tuo file:

```
print(math.pi)          # 3.141592653589793 – il pi greco
print(math.sqrt(16))    # 4.0 – radice quadrata
print(math.floor(3.7))  # 3 – arrotondamento verso il basso
print(math.ceil(3.2))   # 4 – arrotondamento verso l'alto
```

Quando scrivi `math.sqrt`, stai dicendo: "usa la funzione `sqrt` che si trova nel modulo `math`".

Importare solo quello che ti serve con `from ... import`

Se sai già quali funzioni ti servono, puoi importarle direttamente. Così non devi scrivere il nome del modulo ogni volta:

```
from math import pi, sqrt, floor # Importo solo queste tre cose

print(pi)          # 3.141592653589793
print(sqrt(25))    # 5.0
print(floor(4.9))  # 4
```

`from math import *` — sconsigliato

Si può importare tutto in una volta con `*`, ma è una cattiva abitudine:

```
from math import * # Importa TUTTO dal modulo math

print(sin(0))      # Funziona, ma...
```

Il problema: non si capisce più da dove vengono le funzioni. Se due moduli hanno una funzione con lo stesso nome, una sovrascrive l'altra senza avvertirti.

Rinominare con **as**

Puoi dare un soprannome a un modulo o a una funzione con la parola **as**. Utile se il nome è lungo o se vuoi evitare conflitti:

```
print(m.pi)          # 3.14159...

from math import sqrt as radice_quadrata  # Rinomino la funzione
print(radice_quadrata(9))  # 3.0
```

Alcune librerie molto usate hanno soprannomi convenzionali che tutti usano:

Creare il tuo modulo

Un modulo è semplicemente un file **.py**. Crea un file chiamato **calcolatrice.py**:

```
# calcolatrice.py – il mio modulo personale

def somma(a, b):
    return a + b

def sottrai(a, b):
    return a - b

def moltiplica(a, b):
    return a * b

pi = 3.14159
```

Poi in un altro file nella **stessa cartella**:

```
# main.py – uso il mio modulo

print(calcolatrice.somma(3, 4))    # 7
print(calcolatrice.pi)              # 3.14159
```

Il blocco `if __name__ == "__main__"`

Quando Python importa un modulo, esegue tutto il codice al suo interno. Questo può causare problemi se hai del codice di test che non vuoi far girare durante l'importazione.

La soluzione è racchiudere il codice di test in questo blocco speciale:

```
# calcolatrice.py

def somma(a, b):
    return a + b

if __name__ == "__main__":
    # Questo viene eseguito SOLO se avvii calcolatrice.py direttamente
    # NON viene eseguito se qualcuno fa "import calcolatrice"
    print(somma(3, 4))    # Test
```

Regola semplice: `__name__ == "__main__"` è `True` solo per il file che hai avviato tu, non per quelli importati.

Moduli utili già inclusi in Python

Modulo	A cosa serve
<code>math</code>	Funzioni matematiche (radici, trigonometria, logaritmi)
<code>random</code>	Numeri casuali
<code>datetime</code>	Date e orari
<code>os</code>	Interazione con il sistema operativo (file, cartelle)
<code>json</code>	Leggere e scrivere file JSON
<code>re</code>	Espressioni regolari (ricerche avanzate nel testo)
<code>time</code>	Misurare il tempo, mettere in pausa il programma

Il modulo `random` — numeri casuali

```
print(random.random())           # Numero decimale casuale
tra 0.0 e 1.0
print(random.randint(1, 10))     # Numero intero casuale tra
1 e 10 (inclusi)
print(random.choice(["mela", "banana", "uva"])) # Sceglie un elemento a
caso dalla lista

lista = [1, 2, 3, 4, 5]
random.shuffle(lista)            # Mescola la lista (modifica la lista originale)
print(lista)                     # Ordine casuale, es: [3, 1, 5, 2, 4]
```

Il modulo **OS** — interazione con il sistema

```
print(os.getcwd())          # Mostra in quale cartella ti trovi ora
print(os.listdir("."))      # Elenca i file nella cartella corrente
print(os.path.exists("file.txt")) # True se il file esiste, False
                                altrimenti
```

Espressioni regolari

Introduzione alle espressioni regolari con il modulo `re` in Python.

Perché ti servono le espressioni regolari

Immagina di dover controllare se un indirizzo email è scritto correttamente, o di dover trovare tutti i numeri di telefono in un testo. Potresti farlo con decine di condizioni `if`, oppure usare uno strumento molto più potente: le **espressioni regolari**.

Cosa sono le espressioni regolari?

Le **espressioni regolari** (dette anche "regex" o "regexp") sono una mini-lingua per descrivere **scemi di testo**. Servono a:

- Verificare se una stringa corrisponde a un certo formato ("è un'email valida?")
- Trovare scemi in un testo ("tutti i numeri in questo paragrafo")
- Estrarre informazioni ("dammi solo la data da questa riga")
- Sostituire parti di testo ("sostituisci tutti i numeri con `X` ")

In Python si usano con il modulo `re` :

I mattoncini di base: cosa significano i simboli

Simbolo	Significato	Esempio
<code>.</code>	Qualsiasi carattere (eccetto a capo)	<code>a.c</code> trova "abc", "aXc", "a3c"...
<code>\d</code>	Una cifra (0-9)	<code>\d\d</code> trova "42", "07"...
<code>\D</code>	Non una cifra	
<code>\w</code>	Lettera, cifra o underscore	
<code>\s</code>	Spazio, tab o a capo	
<code>^</code>	Inizio della stringa	<code>^Ciao</code> trova solo stringhe che iniziano con "Ciao"
<code>\$</code>	Fine della stringa	<code>fine\$</code> trova solo stringhe che terminano con "fine"
<code>[abc]</code>	Uno tra a, b, c	
<code>[a-z]</code>	Una lettera minuscola qualsiasi	
<code>[0-9]</code>	Una cifra qualsiasi (come <code>\d</code>)	

Quantificatori: quante volte?

Simbolo	Significato
<code>*</code>	Zero o più volte
<code>+</code>	Una o più volte
<code>?</code>	Zero o una volta
<code>{3}</code>	Esattamente 3 volte
<code>{2,5}</code>	Da 2 a 5 volte

Cercare uno schema in una stringa

`re.search()` — cerca ovunque nella stringa

```
testo = "Il mio numero è 334-1234567"

# Schema: 3 cifre, un trattino, 7 cifre
risultato = re.search(r"\d{3}-\d{7}", testo)

if risultato:
    print(f"Numero trovato: {risultato.group()}") # → 334-1234567
else:
    print("Nessun numero trovato")
```

Il prefisso `r` prima delle virgolette (`r"..."`) indica una **raw string** (stringa "grezza"). Senza la `r`, Python interpreterebbe `\d` come una sequenza speciale (come `\n` per andare a capo), stravolgendo il significato della regex. Con la `r`, Python lascia il testo esattamente come l'hai scritto, senza trasformare niente. Usa sempre `r"..."` quando scrivi espressioni regolari.

`re.match()` — cerca solo all'inizio della stringa

```
testo = "Ciao, mondo!"
risultato = re.match(r"Ciao", testo) # La stringa inizia con "Ciao"?

if risultato:
    print("Trovato!")
else:
    print("Non trovato.")
```

`re.findall()` — trova TUTTE le corrispondenze

```
testo = "Ho 3 mele, 5 banane e 2 arance."
numeri = re.findall(r"\d+", testo)  # Trova tutti i gruppi di cifre
print(numeri)  # ['3', '5', '2']
```

Sostituire testo: `re.sub()`

```
testo = "Il prezzo è 10 euro, poi 20 euro."
nuovo = re.sub(r"\d+", "X", testo)  # Sostituisce tutti i numeri con "X"
print(nuovo)  # Il prezzo è X euro, poi X euro.
```

Dividere testo: `re.split()`

Come `str.split()`, ma puoi dividere su uno schema complesso invece che su un singolo carattere:

```
testo = "mela; banana, uva ciliegia"
# Divide su punto e virgola, virgola o spazi (anche multipli)
parti = re.split(r"[;,\s]+", testo)
print(parti)  # ['mela', 'banana', 'uva', 'ciliegia']
```

Estrarre parti specifiche: i gruppi con `()`

Le parentesi tonde creano dei **gruppi** — puoi estrarre separatamente ogni parte del risultato:

```
testo = "Data di nascita: 15/03/2008"
match = re.search(r"(\d{2})/(\d{2})/(\d{4})", testo)

if match:
    giorno = match.group(1)    # Primo gruppo → 15
    mese = match.group(2)      # Secondo gruppo → 03
    anno = match.group(3)      # Terzo gruppo → 2008
    print(f"Giorno: {giorno}, Mese: {mese}, Anno: {anno}")
```

Riutilizzare uno schema: `re.compile()`

Se usi lo stesso schema molte volte, "compilalo" una volta sola per rendere il codice più efficiente:

```
schema_numeri = re.compile(r"\d+")    # Compila lo schema una volta

frasi = ["Ho 3 mele", "Ho 10 banane", "nessun numero qui"]
for frase in frasi:
    trovati = schema_numeri.findall(frase)
    if trovati:
        print(f"Trovati: {trovati}")
```

Esempi pratici

Verificare se un'email è valida

```
def valida_email(email):
    # Schema: caratteri@caratteri.dominio (almeno 2 lettere)
    schema = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"
    return bool(re.match(schema, email))

print(valida_email("alice@esempio.it"))    # True ← valida
print(valida_email("non_valida.com"))      # False ← manca la @
print(valida_email("@dominio.it"))         # False ← manca la parte prima
della @
```

Verificare un numero di telefono italiano

```
def valida_telefono(numero):
    # Schema: opzionalmente +39, poi 9 o 10 cifre
    schema = r"^(\+39)?[0-9]{9,10}$"
    return bool(re.match(schema, numero))

print(valida_telefono("3341234567"))      # True
print(valida_telefono("+393341234567"))   # True
print(valida_telefono("abc"))             # False
```

Classi e oggetti

Introduzione alla programmazione orientata agli oggetti in Python.

Qual è la funzione di classi e oggetti

Immagina di dover gestire le informazioni di 30 studenti in un programma. Senza un modo per organizzare i dati, ti ritroveresti con decine di variabili sparse ovunque: `nome1` , `eta1` , `voto1` , `nome2` , `eta2` , `voto2` ...

Le **classi** risolvono questo problema: ti permettono di raggruppare dati e funzioni correlate in un unico "pacchetto" chiamato **oggetto**.

La differenza tra classe e oggetto

Una **classe** è come il **progetto di un'automobile**: descrive come deve essere fatta, ma non è un'auto vera.

Un **oggetto** è l'**automobile vera**, costruita seguendo quel progetto.

Dal stesso progetto (classe) puoi costruire molte auto (oggetti) diverse — ognuna con il suo colore, la sua targa, il suo chilometraggio.

Definire una classe

```
class Persona:
    pass # "pass" significa: classe vuota per ora, aggiungeremo cose dopo
```

Il costruttore: `__init__`

Il metodo `__init__` è il **costruttore** — viene eseguito automaticamente ogni volta che crei un nuovo oggetto. Serve a impostare i dati iniziali dell'oggetto.

```
class Persona:
    def __init__(self, nome, eta):
        # "self" è l'oggetto che stiamo creando
        # self.nome e self.eta sono i dati dell'oggetto (chiamati
        "attributi")
        self.nome = nome
        self.eta = eta
```

`self` è un riferimento all'oggetto stesso. È sempre il primo parametro di ogni metodo in una classe, anche se quando chiami il metodo non lo scrivi esplicitamente.

Creare oggetti (istanze)

```
alice = Persona("Alice", 16)  # Crea una Persona con nome="Alice" e eta=16
bob = Persona("Bob", 17)      # Crea una Persona diversa

# Accedere ai dati dell'oggetto con il punto
print(alice.nome)  # Alice
print(bob.eta)     # 17
```

`alice` e `bob` sono due **istanze** diverse della stessa classe `Persona`. Hanno gli stessi "campi", ma valori diversi — come due auto dello stesso modello ma con targhe diverse.

Aggiungere metodi

I **metodi** sono funzioni definite dentro la classe. Descrivono le **azioni** che un oggetto può fare.

```

class Persona:
    def __init__(self, nome, eta):
        self.nome = nome
        self.eta = eta

    def saluta(self):
        # Questo metodo usa i dati dell'oggetto (self.nome, self.eta)
        print(f"Ciao, mi chiamo {self.nome} e ho {self.eta} anni.")

    def compleanno(self):
        # Questo metodo modifica un dato dell'oggetto
        self.eta += 1
        print(f"Buon compleanno {self.nome}! Ora hai {self.eta} anni.")

alice = Persona("Alice", 16)
alice.saluta()      # → Ciao, mi chiamo Alice e ho 16 anni.
alice.compleanno()  # → Buon compleanno Alice! Ora hai 17 anni.
alice.saluta()      # → Ciao, mi chiamo Alice e ho 17 anni.

```

Attributi di classe vs attributi di istanza

- **Attributi di istanza:** appartengono al singolo oggetto (ogni oggetto ha il suo valore)
- **Attributi di classe:** sono condivisi da tutti gli oggetti della classe (come una bacheca comune)

```

class Studente:
    scuola = "Liceo Scientifico"  # Attributo di classe: uguale per tutti

    def __init__(self, nome):
        self.nome = nome          # Attributo di istanza: diverso per
ciascuno

alice = Studente("Alice")
bob = Studente("Bob")

print(alice.scuola)  # Liceo Scientifico
print(bob.scuola)    # Liceo Scientifico ← stesso valore per tutti
print(alice.nome)    # Alice
print(bob.nome)      # Bob               ← valore diverso per ciascuno

```

Il metodo `__str__` : come stampare un oggetto

Per default, se fai `print(alice)` Python stampa qualcosa di poco leggibile come `<__main__.Persona object at 0x...>`. Il metodo speciale `__str__` ti permette di decidere tu come appare l'oggetto quando viene stampato.

```
class Persona:
    def __init__(self, nome, eta):
        self.nome = nome
        self.eta = eta

    def __str__(self):
        # Questo viene usato da print() e str()
        return f"Persona(nome={self.nome}, eta={self.eta})"

alice = Persona("Alice", 16)
print(alice)  # → Persona(nome=Alice, eta=16)
```

Esempio completo: la classe Rettangolo

```
class Rettangolo:
    def __init__(self, base, altezza):
        self.base = base
        self.altezza = altezza

    def area(self):
        # Calcola l'area: base × altezza
        return self.base * self.altezza

    def perimetro(self):
        # Calcola il perimetro: 2 × (base + altezza)
        return 2 * (self.base + self.altezza)

    def __str__(self):
        return f"Rettangolo {self.base}x{self.altezza}"

r = Rettangolo(5, 3)
print(r)           # → Rettangolo 5x3
print(r.area())    # → 15
print(r.perimetro()) # → 16

# Puoi creare più rettangoli indipendenti
r2 = Rettangolo(10, 4)
print(r2.area())   # → 40
```

Ogni rettangolo è un oggetto indipendente: cambiare `r` non influisce su `r2`.

Ereditarietà in Python

Come creare classi basate su altre classi in Python.

Perché è utile usare l'ereditarietà

Immagina di dover descrivere gli animali in un programma: cani, gatti, uccelli. Tutti respirano, tutti mangiano — ma ognuno fa un suono diverso. Dovresti riscrivere il codice per "respira" e "mangia" tre volte?

L'**ereditarietà** risolve questo problema: scrivi il codice comune una volta sola in una classe "genitore", e le classi "figlie" lo ereditano automaticamente — aggiungendo solo le loro particolarità.

La metafora della famiglia

Pensa all'ereditarietà biologica:

- Un figlio eredita dai genitori molte caratteristiche (colore degli occhi, gruppo sanguigno)
- Ma ha anche caratteristiche sue proprie
- Può anche avere caratteristiche simili ai genitori ma con variazioni (stesso naso, ma di misura diversa)

In programmazione funziona esattamente così.

Sintassi di base

```
class ClassePadre:
    # metodi e attributi del genitore

class ClasseFiglia(ClassePadre):    # ← il nome tra parentesi indica
    "eredita da"
    # eredita tutto dal padre
    # può aggiungere roba nuova o modificare ciò che eredita
```

Esempio: animali

```
class Animale:
    def __init__(self, nome):
        self.nome = nome

    def respira(self):
        print(f"{self.nome} respira.")

    def mangia(self):
        print(f"{self.nome} mangia.")

# Cane eredita da Animale: ha tutti i suoi metodi + "abbaia"
class Cane(Animale):
    def abbaia(self):
        print(f"{self.nome} fa: Bau bau!")

# Gatto eredita da Animale: ha tutti i suoi metodi + "fa_le_fusa"
class Gatto(Animale):
    def fa_le_fusa(self):
        print(f"{self.nome} fa le fusa...")

fido = Cane("Fido")
fido.respira()      # → Fido respira.    (metodo ereditato da Animale)
fido.mangia()       # → Fido mangia.     (metodo ereditato da Animale)
fido.abbaia()       # → Fido fa: Bau bau! (metodo proprio di Cane)

micio = Gatto("Micio")
micio.respira()     # → Micio respira.
micio.fa_le_fusa()  # → Micio fa le fusa...
```

Sovrascrivere i metodi (Override)

La classe figlia può **ridefinire** un metodo del genitore per cambiarne il comportamento. È come un figlio che ha "lo stesso tipo di carattere" del padre, ma con sfumature diverse.

```
class Animale:
    def __init__(self, nome):
        self.nome = nome

    def suono(self):
        print("...") # Suono generico

class Cane(Animale):
    def suono(self): # Sovrascrive il metodo del genitore
        print(f"{self.nome} fa: Bau bau!")

class Gatto(Animale):
    def suono(self): # Sovrascrive il metodo del genitore
        print(f"{self.nome} fa: Miao!")

fido = Cane("Fido")
micio = Gatto("Micio")

fido.suono() # → Fido fa: Bau bau!
micio.suono() # → Micio fa: Miao!
```

super() : richiamare il genitore

A volte non vuoi **sostituire** completamente un metodo del genitore, ma vuoi **estenderlo** — aggiungere comportamento in più. `super()` ti permette di chiamare il metodo originale del genitore.

```
class Animale:
    def __init__(self, nome, eta):
        self.nome = nome
        self.eta = eta

    def presenta(self):
        print(f"Mi chiamo {self.nome} e ho {self.eta} anni.")

class Cane(Animale):
    def __init__(self, nome, eta, razza):
        super().__init__(nome, eta) # Chiama __init__ del genitore per i
        # dati comuni
        self.razza = razza          # Aggiunge il dato extra "razza"

    def presenta(self):
        super().presenta()          # Prima esegui la presentazione del
        # genitore...
        print(f"Sono un {self.razza}.") # ...poi aggiungi info in più

fido = Cane("Fido", 3, "Labrador")
fido.presenta()
# → Mi chiamo Fido e ho 3 anni.
# → Sono un Labrador.
```

Verificare la parentela tra classi e oggetti

```
class Animale:
    pass

class Cane(Animale):
    pass

fido = Cane()

# isinstance() verifica se un oggetto è di un certo tipo (o di un suo
# "antenato")
print(isinstance(fido, Cane))      # True ← fido è un Cane
print(isinstance(fido, Animale))  # True ← fido è anche un Animale (per
# ereditarietà)

# issubclass() verifica se una classe è "figlia" di un'altra
print(issubclass(Cane, Animale))  # True ← Cane è una sottoclasse di
Animale
```

Esempio pratico: forme geometriche

```
class Forma:
    def __init__(self, colore):
        self.colore = colore

    def descrivi(self):
        print(f"Sono una forma {self.colore}.")

# Rettangolo eredita da Forma
class Rettangolo(Forma):
    def __init__(self, base, altezza, colore):
        super().__init__(colore)      # Passa il colore al genitore
        self.base = base
        self.altezza = altezza

    def area(self):
        return self.base * self.altezza

# Cerchio eredita da Forma
class Cerchio(Forma):
    def __init__(self, raggio, colore):
        super().__init__(colore)      # Passa il colore al genitore
        self.raggio = raggio

    def area(self):
        return 3.14159 * self.raggio ** 2

r = Rettangolo(5, 3, "rosso")
r.descrivi()      # → Sono una forma rosso.    (metodo ereditato da Forma)
print(r.area())   # → 15

c = Cerchio(4, "blu")
c.descrivi()      # → Sono una forma blu.
print(c.area())   # → 50.265...
```

Entrambe le forme condividono il metodo `descrivi()` — scritto una volta sola nel genitore — e ognuna ha la sua formula per calcolare l'area.

Iteratori in Python

Come funzionano gli iteratori in Python.

Perché ti serve conoscere gli iteratori

Ogni volta che scrivi `for elemento in lista`, Python sta usando un meccanismo nascosto chiamato **iteratore**. Capire come funziona ti permette di creare le tue sequenze personalizzate — anche sequenze che non finiscono mai, o che generano i dati "al volo" senza occupare memoria.

La differenza tra iterabile e iteratore

- **Iterabile**: qualcosa su cui puoi fare un ciclo `for`. Liste, stringhe, tuple, dizionari sono iterabili.
- **Iteratore**: un oggetto che "sa dove si trova" nella sequenza e sa come ottenere l'elemento successivo.

Pensa a un libro:

- Il **libro** è l'iterabile (contiene tutti i capitoli)
- Il **segnalibro** è l'iteratore (ricorda a quale pagina sei arrivato)

Come funziona `for` dentro Python

Quando scrivi:

```
for x in [1, 2, 3]:  
    print(x)
```

Python esegue in realtà questi passi:

1. Chiama `iter([1, 2, 3])` per creare un iteratore dalla lista
2. Chiama ripetutamente `next(iteratore)` per ottenere un elemento alla volta
3. Quando la lista è finita, `next()` lancia `StopIteration` e il ciclo si ferma

Puoi vedere questi passi manualmente:

```
lista = [1, 2, 3]
iteratore = iter(lista)    # Crea il "segnalibro"

print(next(iteratore))     # 1 ← prende il primo elemento e avanza
print(next(iteratore))     # 2 ← prende il secondo elemento e avanza
print(next(iteratore))     # 3 ← prende il terzo elemento e avanza
# print(next(iteratore))   # Qui lancerebbe StopIteration: la lista è
# finita!
```

Creare un iteratore personalizzato

Puoi creare una classe che si comporta come un iteratore implementando due metodi speciali:

- `__iter__` : restituisce l'iteratore stesso
- `__next__` : restituisce il valore successivo (o lancia `StopIteration` se è finita)

```

class ContaDa:
    """Un iteratore che conta da 'inizio' fino a 'fine', incluso."""

    def __init__(self, inizio, fine):
        self.corrente = inizio    # Da dove partiamo
        self.fine = fine          # Dove ci fermiamo

    def __iter__(self):
        return self              # L'iteratore restituisce se stesso

    def __next__(self):
        if self.corrente > self.fine:
            raise StopIteration   # Abbiamo finito!
        valore = self.corrente
        self.corrente += 1        # Avanza al prossimo
        return valore

# Ora possiamo usarlo in un ciclo for, esattamente come una lista
for numero in ContaDa(1, 5):
    print(numero)
# → 1
# → 2
# → 3
# → 4
# → 5

```

Generatori: il modo più semplice

Creare una classe iteratore completa funziona, ma è verboso. I **generatori** ti permettono di fare la stessa cosa con molto meno codice.

Un generatore è una funzione che usa `yield` invece di `return`. Ogni volta che viene chiamata, si "ferma" a `yield`, restituisce il valore, e riprende da lì alla chiamata successiva — come un libro che segna automaticamente la pagina.

```
def conta_da(inizio, fine):
    corrente = inizio
    while corrente <= fine:
        yield corrente    # "Ecco il valore, mettiti in pausa finché non ti
        serve il prossimo"
        corrente += 1

# Si usa esattamente come prima
for numero in conta_da(1, 5):
    print(numero)
# → 1, 2, 3, 4, 5
```

La differenza rispetto a `return` : con `return` la funzione finisce e dimentica tutto. Con `yield` la funzione si mette in pausa e ricorda esattamente dove si trovava.

Generator expression: la versione compatta

Come le **list comprehension** (la sintassi `[x**2 for x in range(10)]` che crea una lista in una riga) ma con le parentesi tonde invece di quelle quadre. Producono un generatore invece di una lista: i valori vengono calcolati uno alla volta, solo quando servono, risparmiando memoria.

```
# List comprehension: crea TUTTI i quadrati in memoria subito
quadrati_lista = [x ** 2 for x in range(10)]

# Generator expression: produce UN quadrato alla volta, solo quando serve
quadrati_gen = (x ** 2 for x in range(10))

for q in quadrati_gen:
    print(q)    # → 0, 1, 4, 9, 16, 25, 36, 49, 64, 81
```

Perché usare i generatori? Il problema della memoria

Immagina di dover lavorare con un milione di numeri:

```
# Senza generatore: tutti i numeri vengono creati in memoria subito
# Con 1.000.000 numeri, questo occupa decine di megabyte di RAM
numeri_lista = list(range(1_000_000))

# Con generatore: i numeri vengono creati uno alla volta, solo quando servono
# Occupa quasi zero memoria – indipendentemente da quanti numeri ci sono
numeri_gen = range(1_000_000)
```

I generatori sono come un nastro trasportatore: producono un pezzo alla volta, invece di mettere tutto sul tavolo in una volta.

Strumenti pronti: il modulo **itertools**

Python include una raccolta di iteratori già pronti, molto utili:

```
# Ciclo infinito su una sequenza (utile ad esempio per semafori o animazioni)
colori = itertools.cycle(["rosso", "giallo", "verde"])
for i in range(6):
    print(next(colori))    # → rosso, giallo, verde, rosso, giallo, verde

# Unire più sequenze in una sola
for x in itertools.chain([1, 2], [3, 4], [5]):
    print(x)    # → 1, 2, 3, 4, 5

# Prendere solo i primi n elementi di una sequenza (anche infinita)
for x in itertools.islice(range(1_000_000), 5):
    print(x)    # → 0, 1, 2, 3, 4
```